

ТЕХНОЛОГИЧНО УЧИЛИЩЕ ЕЛЕКТРОННИ СИСТЕМИ

към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

ДИПЛОМНА РАБОТА

по професия код 481020 “Системен програмист”

специалност код 4810201 “Системно програмиране”

Тема: Проектиране и реализация на LLM система GIANT v2 с разширение Anchor-TiDAR за ускорена генерация

Дипломант: Антон Христов

Дипломен ръководител:

СОФИЯ

2 0 2 6

УВОД

През последните години големите езикови модели (Large Language Models, LLM) се превърнаха в основен компонент в съвременните софтуерни системи за търсене, анализ, програмиране и диалогови интерфейси. Паралелно с това нарасна и нуждата от practically usable open implementation, която да позволява не само обучение и инференс, но и инженерна устойчивост: повторяеми експерименти, работа на ограничен брой GPU ресурси, преносимост между локална и отдалечена среда и ясно структурирани артефакти.

Настоящата дипломна работа е фокусирана върху разработката на системата SUPER-GIANT, състояща се от две основни части:

- базова LLM платформа GIANT/v2 (данни, модел, обучение, инференс);
- разширение TiDAR, което използва базовата инфраструктура и надгражда обучението и декодирането със забързан diffusion-ориентиран подход, базиран на научния труд “Think in Diffusion, Talk in Autoregression” от NVIDIA Research и надграждането му със собствени модификации.

Реализацията е изцяло на Python, върху JAX стак, с фокус върху:

- скалируем data pipeline с шардове и лесна интеграция с големи хъбове за данни като Huggingface или Kaggle;
- стабилно обучение с checkpoint/resume и възстановяване на състоянието на data loader;
- ускорен KV-cache инференс;
- инженерни практики за контейнеризация, синхронизация със S3-съвместимо хранилище и работа с GPU в облака.

Целта на дипломната работа е да се проектира и реализира цялостен програмен продукт за обучение и използване на езиков модел, който едновременно:

- осигурява стабилна базова архитектура за autoregressive обучение (GIANT/v2);
- позволява бързо експериментиране с нови training objectives и decoding схеми (TiDAR);
- остава практически приложим в реални инженерни условия (CI/CD, remote execution, cache/sync, reproducibility).

Основни задачи на дипломната работа:

- проучване на съществуващи LLM системи и инструменти;
- описание на архитектурата, алгоритмите и вътрешния модел на данните;
- реализация на ключовите модули в GIANT/v2 и TiDAR;
- валидиране чрез експерименти, логове, benchmark артефакти и потребителски сценарии.

ПЪРВА ГЛАВА

ПРЕГЛЕД НА СЪЩЕСТВУВАЩИ СИСТЕМИ, ПОДОБНИ НА GIANT, И ИЗВЕСТНИ РАЗВОЙНИ СРЕДСТВА

В тази глава са разгледани решения и инструменти, близки до тематиката на дипломната работа: обучение и ускорен инференс на decoder-only езикови модели.

1.0 Термини и технологии (кратки дефиниции)

За целите на дипломната документация по-долу са дадени кратки дефиниции на основните термини. След първото им обяснение те се използват в текста и на английски.

- autoregressive (AR) - режим, при който моделът генерира следващ token на база на предходните токени;
- inference - процесът на използване на вече обучен модел за генериране/предсказване, без update на параметрите;
- KV cache - кеш за key/value представянията в attention слоя, който ускорява decode стъпките;
- prefill - начален forward pass върху prompt-a, който запълва KV cache преди същинския decode;
- decode - итеративна фаза на генериране на нови токени след prefill;
- sharding - разделяне на голям dataset на по-малки shard файлове за по-ефективно четене;
- checkpoint - запис на състоянието на модела и оптимизатора в конкретна training стъпка;
- resume - продължаване на training от съществуващ checkpoint вместо start от нула;
- stage - отделен обучителен етап с конкретни настройки (данни, дължина, епохи, loss коефициенти);
- curriculum - планирана последователност от stages, обикновено от по-лесни към по-сложни режими;
- loss function - функция, която измерва грешката на модела и определя посоката на оптимизация;
- optimizer - алгоритъм за обновяване на параметрите по градиент (например AdamW);
- batch - група от примери, обработвани заедно в една training стъпка;
- epoch - едно пълно преминаване през тренировъчния набор данни;
- throughput - скорост на обработка (например tokens/s);
- latency - време за единична операция/отговор (например време за decode итерация);
- benchmark - стандартизирано измерване на производителност при фиксирани условия;
- artifact - генериран резултат от pipeline/training/inference (лог, checkpoint, графика, отчет);
- Docker - контейнерна среда за възпроизводимо изпълнение на едни и същи зависимости;
- CUDA - GPU изчислителна платформа на NVIDIA, използвана за ускорение на ML изчисленията;
- JAX - фреймуърк за числени изчисления с автоматично диференциране и JIT компилация;
- Tailscale - private mesh VPN за защитен remote достъп между машини;
- S3 - обектно хранилище (cloud/local-compatible) за данни, checkpoints и синхронизация между среди.

LLaMA семейство (Meta)

LLaMA моделите се утвърдиха като де факто стандарт за отворени foundation модели. Основните им предимства са добра scaling стратегия, стабилни архитектурни избори (RMSNorm, RoPE, SwiGLU) и голяма екосистема от производни реализации. За проекта GIANT тези модели са референтна точка за архитектурни и обучителни практики.

SmolLM / компактни open модели

SmolLM профилите са важни като practical baseline за експерименти с ограничен GPU бюджет. В TiDAR частта на проекта се използват точно такива размери (135M и 360M), което позволява многократни сравнения между loss конфигурации и decode варианти в разумно време.

nanoGPT / minGPT

Тези проекти показват минимална, ясно четима реализация на autoregressive Transformer. Те са полезни за концептуално разбиране, но не покриват production-like нуждите от sharding, resume, инфраструктура за отдалечен GPU и систематични benchmark отчети.

Megatron-LM и distributed тренировка

Megatron-LM е ориентиран за мащабно обучение с моделна/данна паралелизация. В настоящата работа не се гони multi-node distributed setup; фокусът е единичен GPU workflow с максимална инженерна практичност. Въпреки това принципите за разделяне на data/model/optimizer отговорности са сходни.

vLLM и TensorRT-LLM

Тези системи са водещи при serving/latency оптимизации и силно ефективно управление на KV-cache. В GIANT/TiDAR част от идеите за cache политика и throughput benchmarking са приложени в custom JAX стек, като изследователският фокус е върху Anchor-TiDAR декодиране и loss-driven подобрения.

Python

Изборът на Python е определен от бързия цикъл за изследователска разработка, широка библиотечна поддръжка и добра интеграция с ML инструменти, cloud execution и автоматизация.

JAX + Flax

JAX е избран заради XLA компилацията, контрол върху устройството и възможност за високопроизводителни JIT графи. Flax осигурява структурирано model definition API и variable collections, критични за KV-cache и за чисто разделяне на train/inference състояние.

Optax + Orbx

Optax е използван за optimizer pipeline (clip + AdamW + scheduler), а Orbx - за асинхронни mini checkpoint-и. Тази комбинация подобрява устойчивостта при long-running експерименти и намалява риска от загуба на прогрес при прекъсване.

HuggingFace Hub / Datasets / Transformers

Data pipeline частта използва datasets за ingest на публични корпуси (streaming и non-streaming режими), а transformers - за tokenizer интеграция. Това позволява стабилен и възпроизводим preprocess workflow към Arrow шардове.

Docker, Tailscale и S3 инструменти

Инфраструктурният слой в CICD/ дава:

- Docker образи с готов CUDA/JAX стек;
- Tailscale-based secure remote достъп;
- S3 операции чрез s5cmd wrapper;
- скриптове за бърз sync на локални промени към remote GPU среда.



Figure 1: Контейнерна среда за reproducible изпълнение

ВТОРА ГЛАВА

ФУНКЦИОНАЛНИ ИЗИСКВАНИЯ, АРГУМЕНТАЦИЯ НА ИЗБОРА НА СРЕДСТВА И ФОРМАЛЕН МОДЕЛ

Тази глава описва какво трябва да прави системата, защо е избран точно този технологичен стек и как е организиран вътрешният модел на данните и модулите. Целта е читателят да може да проследи логиката от входните данни до крайния training и inference процес, дори без предишен опит с LLM инфраструктура.

2.1 Функционални изисквания

В контекста на тази работа **обучение** (training) означава итеративно нагласяване на параметрите на модела така, че разликата между очакван и предсказан изход да намалява с всяка следваща стъпка. Това става чрез изчисляване на грешка (loss), обратно разпространение на градиентите и обновяване на параметрите от оптимизатор.

2.1.1 Входни източници и ingest pipeline

Системата трябва да поддържа ingest (контролирано зареждане и уеднаквяване) на текстови корпуси от няколко типа източници:

- публични HuggingFace datasets;
- локални JSON/JSONL директории;
- chat-структурирани данни с role/content полета.

Ingest pipeline е последователност от стъпки, която превръща суровия текст в формат, удобен за обучение: нормализация на текст, токенизация, филтриране и запис в стандартизирани shard файлове.

2.1.2 Подготовка на корпус, токенизация и stage-и

Токенизация означава разбиване на текста на по-малки единици (токени), които моделът реално обработва като числа. Вместо да “чете” директно думи, моделът работи с последователности от token ID стойности.

Системата използва **stages** (обучителни етапи). Всеки stage може да има собствена дължина на последователността (seq_len), дял от данните (fraction) и брой епохи (epochs).

Епоха (epoch) е едно пълно преминаване през избраната част от dataset-а за дадения stage.

2.1.3 Конфигурационно управление на експерименти

Експериментът се описва чрез YAML конфигурации. Това е важно за възпроизводимостта: при същите конфигурации и данни резултатът трябва да бъде максимално близък.

Основните нива са:

- глобални настройки в Global_Config.yml (пътища, базова среда);
- локални model/training настройки в Config.yml;
- stage-level настройки за dataset дялове и loss коефициенти;
- CLI overrides за бързи промени без пренаписване на конфигурационните файлове.

2.1.4 Мониторинг и метрики по време на изпълнение

При дълги run-ове е задължително системата да показва междинно състояние, а не само краен резултат.

Затова се изисква:

- логване на loss и помощни метрики по стъпки;
- acceptance метрики при TiDAR decode;
- запис на checkpoint metadata (стъпка, конфигурация, време);
- автоматично генериране на benchmark отчети.

Метрика е измерима стойност за качество или производителност. Типични примери са loss, accuracy, tokens/s (throughput) и latency.

2.1.5 Управление на хиперпараметри и decode политика

Хиперпараметри са настройки, които не се научават автоматично от модела, а се задават от разработчика: размер на embedding, брой слоеве, размер на batch, learning rate и др.

В тази система управлението обхваща:

- архитектурни параметри (embedding_size, num_heads, num_kv_heads, num_layers);
- числен тип на изчисленията (compute_dtype, param_dtype);
- контекстна дължина и политика за KV cache;
- TiDAR decode параметри (draft_length, attention bias, sampling настройки).

2.1.6 Артефакти, checkpoint-и и отчетност

Артефакт е всеки запазен резултат от изпълнението: файл с параметри, лог, графика, отчет.

Системата трябва да съхранява и поддържа:

- dataset shard-ове + manifest + статистики;
- checkpoint-и на параметри + optimizer/dataloader state;
- inference и benchmark артефакти;
- текстови и визуални отчети (.md, .typ, .pdf).

Checkpoint е снимка на състоянието на обучението в конкретна стъпка. **Resume** е продължаване на обучението от тази снимка, вместо нов старт от нула.

2.2 Аргументация за избор на езика и софтуерните средства

2.2.1 Избор на езика за програмиране

Изборът на Python е обусловен от практичността му за ML инженеринг:

- много богата екосистема за data/ML;
- бърз цикъл на прототипиране;
- ясна интеграция със shell автоматизация, Docker и cloud workflow.

2.2.2 Избор на платформа за разработка

Избрана е GPU-ориентирана среда, защото training и inference на LLM са изчислително тежки и се ускоряват многократно от графични процесори.

Понеже локалната машина не е достатъчна за всички сценарии, работният модел е хибриден:

- локална разработка и подготовка на данни;
- remote GPU изпълнение за тежки run-ове;
- синхронизация чрез Docker + S3 tooling.

Така се постига баланс между цена, скорост и контрол на средата.

2.2.3 Избор на фреймуърк за интерфейс

Системата е CLI-first, т.е. управлява се предимно през команди и конфигурационни файлове, без графичен интерфейс.

Това решение е уместно за изследователски проект, защото дава:

- директен контрол на параметрите;
- лесна автоматизация чрез скриптове;
- добра работа в remote shell среди (ssh/tmux).

2.2.4 Избор на изчислителен фреймуърк

JAX е избран заради JIT компилацията и оптимизациите от XLA. На практика това позволява една Python функция да се компилира до силно оптимизиран изпълним граф за конкретни размерности на входа.

Flax е надстройка над JAX за модулно описание на невронни архитектури и управление на параметри/състояние.

Комбинацията JAX + Flax е ключова за проекта, защото позволява:

- висока производителност;
- ясен модел на train/inference state;
- интеграция с custom KV cache логика и TiDAR маски.

2.2.5 Избор на optimizer/checkpoint инструменти

Optax е използван за optimizer/scheduler логиката, а Orbx - за checkpointing.

Това е важно при дълги обучения, където прекъсване без checkpoint би означавало загуба на часове или дни изчисления.

2.2.6 Избор на средства за deployment и синхронизация

Използват се Docker, Tailscale и S3 съвместими инструменти.

- Docker осигурява еднаква изпълнима среда (еднакви зависимости) между машини;
- Tailscale осигурява защитена частна мрежа за remote достъп;
- S3 workflow позволява надеждна синхронизация на данни и артефакти между локална и remote среда.

2.3 Структура на данните и вътрешен модел на системата

2.3.1 Формат на данните и логика на съхранение

Системата използва файлово-ориентиран модел, а не релационна база данни.

Основният формат за обучителни данни са Arrow shard файлове, придружени от manifest и статистики. Този избор е направен заради бързо последователно четене от data loader-a и добра съвместимост с големи обеми текст.

JSON/TXT формати се използват като входни или междинни формати в ingest етапа, но не и като финален training storage backend.

2.3.2 Stage конфигурация и curriculum модел

Curriculum в този контекст означава планирана последователност от training stages, при която различните етапи имат различни цели и настройки.

```
@dataclass
class StageConfig:
    name: str
    dataset: str
    seq_len: int
    epochs: int
    end_ratio: float
```

Този тип структура позволява прозрачно управление на сложни training сценарии без ръчно пренаписване на кода за всеки нов run.

2.3.3 Конфигурационни домейни на системата

Конфигурацията е разделена на логически домейни:

- model - архитектура на мрежата;
- optimizer - стратегия за update;
- training - batch, stages, logging, checkpointing;

- inference - decode режим и runtime параметри;
- tidar - специфични настройки за TiDAR.

Това разделение улеснява дебъгването и прави експериментите по-проследими.

2.3.4 Схема на dataset записите в shard

Единичният запис в Arrow shard е фиксиран по структура и съдържа:

- input_ids - токените;
- length - реална дължина на последователността;
- loss_mask - позиции, които участват в loss изчислението.

StageDataLoader преобразува тези полета към моделно удобен batch формат (input/target/mask).

2.3.5 Източници и ingest адаптери

StageSourceCfg описва типа източник и правилата за четене (plain text, JSON, chat формат, streaming/non-streaming).

Така един и същ ingest pipeline може да обработва различни входни формати без отделни, несъвместими кодови пътища.

2.4 Цялостна архитектура и структурна организация

Архитектурата е разделена на три слоя:

- интерфейсен слой (CLI + YAML конфигурации);
- управляващи модули (pipeline, training, checkpointing, inference);
- математически модел (Transformer + TiDAR разширения).

2.4.1 Интерфейсен слой (CLI)

Потребителят работи чрез entry point скриптове и конфигурационни файлове. Това гарантира, че всеки run е възпроизводим и може да бъде повторен от друг разработчик при същите условия.

2.4.2 Управляващи модули

Управляващите модули са “гръбнакът” на системата:

- data pipeline модул за build на корпус и shard-ове;
- training оркестратор за stages, optimizer стъпки и логване;
- checkpoint мениджър за save/restore/resume;
- inference runtime за prefill/decode и performance отчети.

TiDAR надгражда тази основа със специализирани маски и допълнителни loss механизми, без да нарушава базовия workflow.

2.4.3 Основни entry points

Ключовите скриптове са:

- GIANT/v2/data_pipeline/build_corpus.py
- GIANT/v2/model/Run_training.py
- GIANT/v2/model/Generate_faster.py
- GIANT/v2/model/jit_inference.py
- TiDAR/model/Run_training.py
- TiDAR/model/inference.py

Така системата запазва ясна граница между базова платформа (GIANT/v2) и изследователски разширения (TiDAR).

2.5 Основен математически модел на трансформера

Тази секция представя интуитивно и формално как работи Transformer архитектурата, върху която е изграден GPT. Подходът е от “идея към формула”: първо какво прави моделът на човешки език, после как се записва математически.

2.5.1 Теоретична отправна точка: Attention Is All You Need

Класическите по-стари езикови модели обработват текста последователно, символ по символ. Това е коректно, но ограничава паралелизма и забавя обучението.

Transformer променя това чрез **attention** механизъм. Интуитивно, attention означава: когато моделът обработва дадена дума, той изчислява кои други думи от контекста са най-важни за нея и им дава по-голяма тежест.

Може да се мисли за attention като “умно претегляне на контекста”:

- моделът сравнява текущата позиция с всички предишни позиции;
- оценява колко е релевантна всяка от тях;
- комбинира информацията според тези оценки;
- предава получената смес към следващите слоеве.

Точно тази идея позволява паралелна обработка на цели блокове от токени и е фундаментална за съвременните LLM системи.

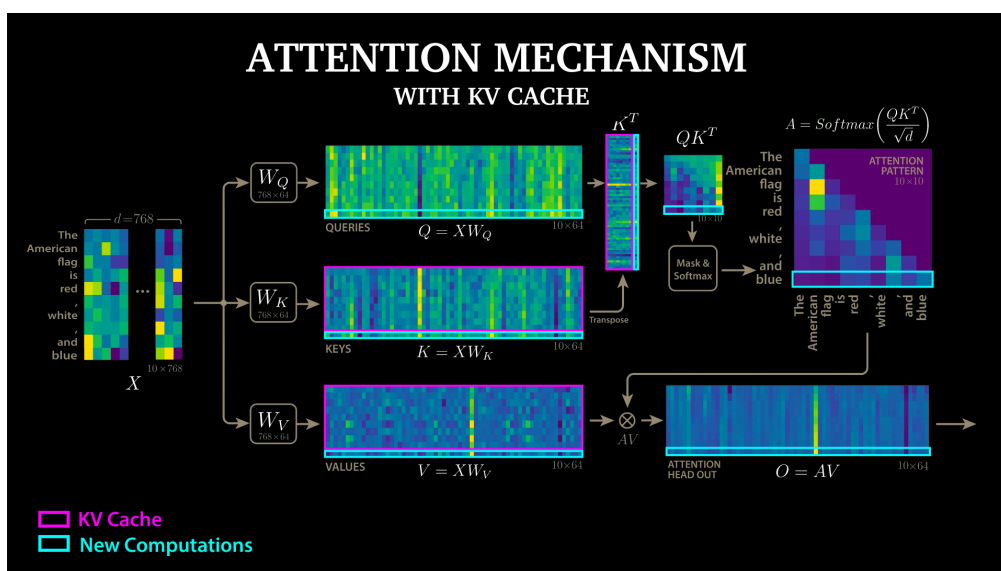


Figure 2: Интуитивна схема на attention механизма

В математически вид attention операторът е:

$$A(Q, K, V) = S\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

където:

- Q (queries) задава какво “търси” текущата позиция;
- K (keys) описва с каква информация разполагат позициите в контекста;
- V (values) е реалното съдържание, което се смесва;
- M е маска (напр. causal маска за decoder режим);
- S е softmax функцията за нормализиране на тежестите.

Softmax се дефинира като:

$$S(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Така weights се интерпретират като вероятностно разпределение (сборът им е 1), което прави смесването стабилно и интерпретируемо.

Класическата форма от оригиналната публикация е:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

В реалните decoder-only модели почти винаги има маска, затова в инженерната форма се използва и M.

2.5.2 Multi-head attention: защо са нужни “много глави”

Ако attention е само един, моделът гледа контекста от една “перспектива”. Multi-head attention въвежда няколко паралелни attention подпространства, които могат да уловят различни типове зависимости едновременно:

- синтактични (напр. кой е подлог/сказуемо);
- семантични (напр. кой обект се отнася към кого);
- по-дълги контекстни връзки (препратки между отдалечени токени).

Формално:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

където:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Компактно:

$$H(Q, K, V) = [h_1 \parallel \dots \parallel h_n]W^O, h_i = A(QW_i^Q, KW_i^K, VW_i^V)$$

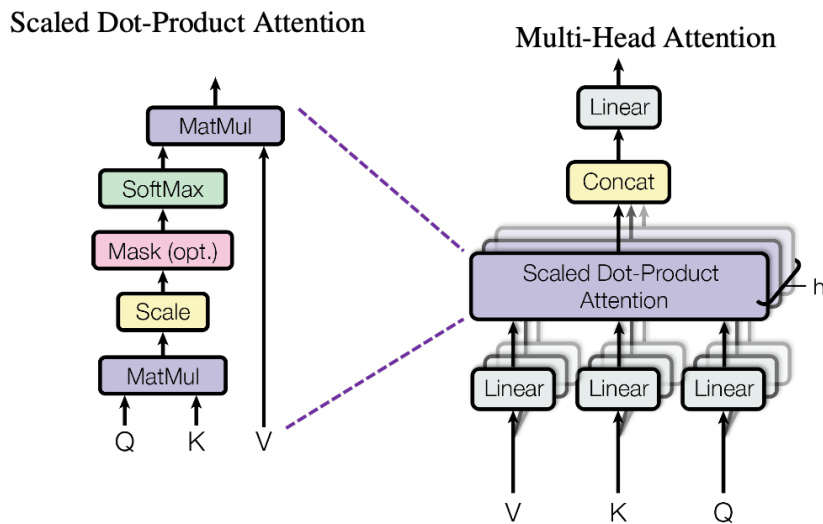


Figure 3: Multi-head attention: блокова схема и формули

2.5.3 От класически Transformer към decoder-only LLM

Оригиналният Transformer е encoder-decoder архитектура. За генеративни езикови модели като GPT се използва **decoder-only** вариант, при който моделът предсказва следващ token от предишните.

Ключовите принципи са:

- **causal masking** - моделът няма право да “гледа в бъдещето”;
- **autoregressive objective** - целта е следващият токен;
- еднопосочен поток при генерация.

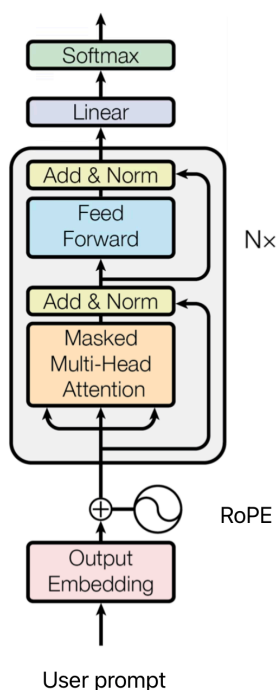
Един decoder блок може да се запише така:

$$x' = x + H(N(x), N(x), N(x))$$

$$y = x' + F(N(x'))$$

където:

- N е нормализация;
- H е multi-head attention;
- F е feed-forward модул;
- добавянето на x и x' са residual връзки за стабилен градиентен поток.



Фигура: Decoder-only Transformer архитектура

Диаграмата показва пътя: входни токени -> embeddings -> поредица от Transformer блокове -> финална нормализация -> логити за следващ токен.

2.5.4 Нормализация, активации и позиционна информация в GIGANT

GIGANT използва съвременен decoder-only профил:

- RMSNorm вместо класически LayerNorm;
- SwiGLU в feed-forward частта;
- RoPE (rotary positional embeddings) за позиционна информация.

RMSNorm:

$$R(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} * g$$

Интуитивно, RMSNorm държи мащаба на активациите стабилен между слоевете, което подпомага обучението при по-дълбоки модели.

SwiGLU гейтинг:

$$G(x) = s(xW_u) * (xW_v), \quad s(t) = t\sigma(t)$$

Интуитивно, това е “контролиран пропуск” на информация: част от каналите се усилват, част се потискат според контекста.

RoPE кодира позицията директно в attention представянията и поддържа по-стабилно поведение при по-дълги контексти.

2.5.5 Обучителна цел (AR) и връзка с TiDAR

Базовата обучителна цел на GIANT е autoregressive cross-entropy:

$$L_1 = -\frac{1}{N} \sum_{b,t} m_{b,t} \log p_{\theta}(x_{b,t+1} | x_{b,\leq t})$$

Тук моделът се наказва, когато вероятността за реалния следващ token е ниска. С много training стъпки това води до постепенно подобрение на next-token предсказването.

Примерен изход от GIANT v2 checkpoint, трениран върху корпус със силно Wikipedia присъствие (заедно с други източници), е:

User: What is the capital of France?

Assistant: The capital of France is sometimes called Paris.<E0S>

- размер на модела: приблизително 101 милиона параметъра;
- обем на обучението: приблизително 2 милиарда токена.

За мащабно сравнение:

- Llama 3 70B - trained on 15T tokens;
- DeepSeek R1 671B - trained on 14.8T tokens.

TiDAR стъпва директно върху тази основа. Тоест първо се учи стабилен autoregressive модел, а след това се добавят TiDAR разширенията (masking логика и допълнителни loss термини) за по-ефективен decode.

2.6 Спекулативно декодиране (въведение преди TiDAR)

Преди представянето на TiDAR е важно да се въведе идеята за speculative decoding, защото именно тя е базата, върху която стъпва по-късното Anchor-TiDAR разширение.

Класическото autoregressive декодиране генерира по един token на стъпка. Това е коректно, но често не използва оптимално паралелните възможности на GPU при inference.

Спекулативното декодиране цели да увеличи throughput, като предлага предварителни токени (draft), които после се верифицират и частично или изцяло се приемат. Така за една итерация може да се валидират няколко бъдещи позиции вместо само една.

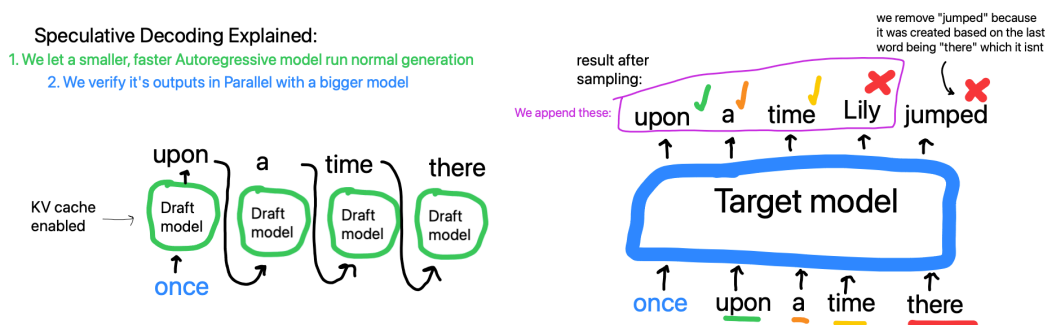


Figure 4: Класически speculative decoding: draft + verify схема

Основната идея е:

- draft е бърза хипотеза за следващите токени;
- verify проверява хипотезата спрямо по-точен/референтен изход;
- приемането на префикс от draft-а води до повече от +1 token прогрес на итерация.

Тази схема е важна за проекта, защото:

- дава теоретична рамка за ускорение на генерацията;
- естествено се комбинира с KV cache;
- подготвя прехода към TiDAR, където verify и predraft логиката се реализират в един специализиран forward pass.

Важно разграничение: класическите speculative decoding методи обикновено използват втори, по-малък draft модел (например по-малка Llama спрямо по-голям verify модел, напр. 3B срещу 70B) или добавят допълнителни архитектурни компоненти върху базовия модел (например допълнителни глави/слоеве, както при EAGLE).

2.6.1 Защо single-user decode не използва добре GPU

При decode с един потребител и малък effective batch GPU често е ограничен от memory bandwidth, а не от чиста аритметична мощност. Понеже един достъп до HBM/GDDR паметта е сравнително скъп, целта е с едно четене на параметри (например K/V матрици) да се направят повече умножения върху повече токени, т.е. по-голям ефективен batch и по-висока заетост на Tensor Core блоковете.

В проекта се използва и FlashAttention (cuDNN backend), което намалява IO overhead в attention стъпката и допълнително подпомага този подход.

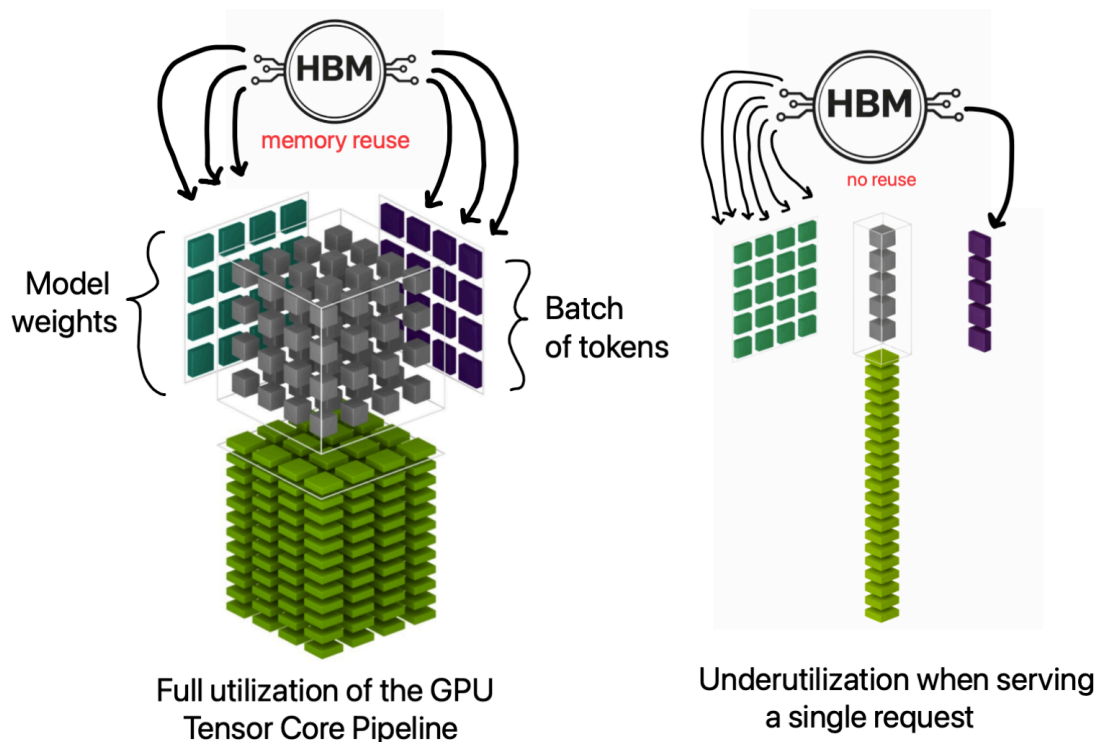


Figure 5: Tensor Core utilization при memory-bound single-user decode

Тази мотивация е важна за разбирането на TiDAR и феномена “Free Token Slots”, където допълнителната паралелна работа в една decode стъпка се използва по-ефективно.

ТРЕТА ГЛАВА

РЕАЛИЗАЦИЯ НА ПРИЛОЖЕНИЕТО

3.1 Реализация на потребителски интерфейс

3.1.1 Основни входни точки (CLI)

Системата е CLI-first и основният интерфейс е набор от entry points:

- GIANT/v2/data_pipeline/build_corpus.py
- GIANT/v2/model/Run_training.py
- GIANT/v2/model/Evaluate.py
- GIANT/v2/model/Generate_faster.py
- GIANT/v2/model/Chat.py
- TiDAR/model/Run_training.py
- TiDAR/model/inference.py

Тези скриптове формират основния интерфейс на системата и покриват пълния цикъл: data prep, обучение, оценка и инференс.

3.1.2 Наблюдение и визуализация на резултати

Визуализацията се реализира чрез логове и експериментални отчети:

- training логове (loss, ppl, accept, greedy_accept);
- checkpoint metadata (train_loss, val_loss, val_ppl);
- Typst отчети с throughput и policy сравнения.

Този подход е избран целенасочено за terminal/vim workflow и за лесно remote наблюдение през SSH.

3.1.3 Управление на stage прогреса

“Времевата линия” в проекта е stage curriculum. Прогресът се движи по stages с различен dataset и контекстна дължина, като се пази възстановимо състояние за всеки етап.

3.1.4 Управление на параметри

Управлението на параметрите е изнесено в YAML конфигурации и CLI flags. Това позволява силно контролирани експерименти с минимални кодови промени.

3.1.5 Добавяне на входни корпуси

Pipeline-ът поддържа HuggingFace и локални JSON/JSONL входи, chat formatting и loss-mask логика.

3.1.6 Експорт на артефакти

Експорт на артефакти към файловата система/S3:

- Arrow shards;
- manifests/statistics;
- checkpoints;
- benchmark logs;
- Typst/PDF отчети.

3.1.7 Примерен training run в терминал

Следният пример показва типично стартиране на обучение през CLI интерфейса:

```
python GIANT/v2/model/Run_training.py \  
  --config GIANT/v2/model/Config.yml \  
  --global_config GIANT/v2/Global_Config.yml \  
  --checkpoint_dir giant-data/GIANT/v2/checkpoints \  
  --data_dir giant-data/GIANT/v2/data
```

```
--checkpoint_every 500 \  
--scan_chunk 8
```

Пример за възстановяване на последната налична checkpoint точка:

```
python GIANT/v2/model/Run_training.py \  
--config GIANT/v2/model/Config.yml \  
--global_config GIANT/v2/Global_Config.yml \  
--checkpoint_dir giant-data/GIANT/v2/checkpoints \  
--resume latest
```

Примерен терминален изход при старт от нула (съкратен):

```
[loader] loading shards from /proj/giant-data/GIANT/v2/processed/stage_base  
...  
[loader] stage=base rows=12800000/12800000 ctx=1024 steps_per_epoch=6250  
  
→ Stage base: seq_len=1024 epochs=1 steps=6250 (resume at 0)  
step    100/10449 | stage base          loss 4.8921 ppl 133.21 (18.4s)  
step    200/10449 | stage base          loss 4.6017 ppl 99.66 (17.9s)  
...  
step    500/10449 | stage base          loss 4.2143 ppl 67.64 (17.2s)  
[metadata] Wrote train_loss=4.214300 to checkpoint .../params/step_0000500.npz  
📁 checkpoint → .../params/step_0000500.npz
```

Примерен терминален изход при --resume latest (примерно от стъпка 5432):

```
↪ Resumed from mini checkpoint at step 5432  
► Restored dataloader state at stage 0 step 5432  
  
→ Stage base: seq_len=1024 epochs=1 steps=6250 (resume at 5432)  
step    5500/10449 | stage base          loss 3.9981 ppl 54.50 (16.9s)  
...  
→ Stage wiki: seq_len=1024 epochs=1 steps=4199 (resume at 0)  
step    6400/10449 | stage wiki          loss 3.7129 ppl 40.97 (16.8s)  
...  
✓ Training complete. Final checkpoint: .../params/step_0010449.npz
```

Примерен multi-stage TiDAR training run с различни loss конфигурации по етапи:

```
python TiDAR/model/Run_training.py \  
--config TiDAR/model/training_configs/Greedy_exp_135m.yml \  
--global_config TiDAR/Global_Config.yml \  
--checkpoint_dir giant-data/TiDAR/checkpoints \  
--checkpoint_every 400 \  
--resume latest
```

```
[loader] loading shards from /proj/giant-data/TiDAR/processed/stage_base  
...  
[loader] stage=base rows=6200000/6200000 ctx=1024 steps_per_epoch=3027  
[loader] loading shards from /proj/giant-data/TiDAR/processed/stage_chat  
...  
[loader] stage=chat rows=1800000/1800000 ctx=1024 steps_per_epoch=878  
  
→ Stage base: seq_len=1024 epochs=1 steps=3027 (resume at 0)
```



```

step      1/3905    | stage base          loss 5.1023 ar 4.8122 diff 5.2761
accept 0.117 greedy_acc 0.081 (19.2s)
step     200/3905    | stage base          loss 4.4311 ar 4.1926 diff 4.5214
accept 0.223 greedy_acc 0.147 hard 3.8872 (17.4s)
...

→ Stage align_kl: seq_len=1024 epochs=1 steps=1000 (resume at 0)
step     3200/3905    | stage align_kl      loss 3.9027 ar 3.7710 diff 3.9488
accept 0.356 greedy_acc 0.244 kl_fwd 0.1821 kl_rev 0.0964 distill 0.1418 topk 0.0889
(16.9s)
step     3600/3905    | stage align_kl      loss 3.7814 ar 3.6562 diff 3.8241
accept 0.391 greedy_acc 0.269 kl_fwd 0.1499 kl_rev 0.0792 distill 0.1184 topk 0.0711
(16.6s)
...

→ Stage chat: seq_len=1024 epochs=1 steps=878 (resume at 0)
step     3900/3905    | stage chat          loss 3.7442 ar 3.6214 diff 3.7811
accept 0.405 greedy_acc 0.281 hard 3.1027 distill 0.1021 (16.4s)

[metadata] Wrote train_loss=3.744200 to checkpoint .../params/step_0003905.npz
✓ Training complete. Final checkpoint: .../params/step_0003905.npz

```

3.2 Реализация на основните системни модули

3.2.1 ConfigLoaderManager

Конфигурациите се зареждат чрез `merge` на глобален и локален `YAML`, след което се резолват относителните пътища спрямо `data_root`. Това осигурява еднакъв кодов път за локално и `remote` изпълнение.

```

def load_configs(config_path=None, global_config_path=None):
    local_cfg = OmegaConf.load(config_path)
    global_cfg = OmegaConf.load(global_config_path)
    cfg = OmegaConf.merge(global_cfg, local_cfg)

    # Унифициране на пътищата спрямо data_root
    cfg.paths.processed_data_root = resolve_path(cfg.paths.processed_data_root,
    cfg.paths.data_root)
    cfg.paths.logs_root = resolve_path(cfg.paths.logs_root, cfg.paths.data_root)
    return cfg

```

3.2.2 TrainLoopOrchestrator

`Run_training.py` реализира `scan`-базиран `training loop` с `gradient accumulation`, `finite checks` и `checkpoint` политика.

3.2.2.1 Основен JIT training loop (`_run_chunk + lax.scan`)

Сърцето на изпълнението е `_run_chunk(...)`, където `batch`-овете се обработват със `lax.scan`, пресмятат се `loss/grad` стойности и се правят проверки за `non-finite` стъпки.

`lax.scan` е `JAX` примитив за “компилиран `for`-цикъл”: вместо `Python` да `dispatch`-ва всяка стъпка поотделно, целият цикъл се представя като една `JIT`-оптимизирана граф операция с `carry` състояние (например `params`, `opt_state`, `accum_grads`, `accum_count`) и последователност от изходи (`losses`). Това намалява `host overhead` и дава по-стабилна производителност при дълги `training run`-ове.

3.2.2.2 Обработка на сигнали и контролирано спиране

Скриптът обработва `SIGINT/SIGTERM` и спира контролирано след текущия `chunk`, за да не се загуби междинно състояние. Практически се поддържат два слоя на устойчивост:

- големи checkpoint-и (params + optimizer + dataloader state), които са “стабилните” запазвания и се пазят;
- мини checkpoint-и, които се записват периодично за бързо възстановяване при внезапно прекъсване (preemption, срив, рестарт).

Така при неочаквано прекъсване може да се продължи от почти последната стъпка, а при нормален stop/етапен преход остават и пълните checkpoint-и за дългосрочно съхранение и проследимост. В практиката training run-овете се поддържат през tmux (включително в CI/CD workflow), което позволява стабилна сесия, лесно повторно закачане към машината и безопасно дълго изпълнение.

3.2.2.3 Gradient accumulation и update политика

Обучението поддържа gradient_accumulation; update се прилага само при достигане на зададения accumulation праг, а остатъчните градиенти се доизчистват при край на stage.

```
# Натрупване на градиенти
accum_grads = tree_map(lambda a, g: a + g, accum_grads, grads)
accum_count = accum_count + 1

# Update само когато достигнем gradient_accumulation
should_update = accum_count == grad_accum
if should_update:
    grads_scaled = tree_map(lambda g: g / grad_accum, accum_grads)
    updates, opt_state = optimizer.update(grads_scaled, opt_state, params)
    params = optax.apply_updates(params, updates)
    accum_grads = tree_map(jnp.zeros_like, accum_grads)
    accum_count = 0
```

На практика това е логика за “мини batch-ове” на ниво update. Вместо целият глобален batch да се побере наведнъж в паметта, той се разделя на няколко по-малки стъпки (micro-batches), които GPU може да обработи с наличния VRAM. Градиентите от тези micro-batches се натрупват и optimizer update се прави накрая, така че се получава ефект на по-голям глобален batch без изискване за голяма еднократна памет.

Този подход е особено важен при по-слаби GPU конфигурации и може да се разшири директно към multi-GPU обучение, където освен локалното accumulation се добавя и синхронизация/редукция между устройствата преди финалния update.

3.2.2.4 Периодично логване на метрики

На всеки log_every стъпки се записват global_step, stage, loss и допълнителни показатели (напр. ppl, accept, greedy_acc, KL/hard/distill/top-k при TiDAR).

Тези метрики се пазят в лог файлове и checkpoint metadata, за да могат по-късно да се използват за сравнение между run-ове, диагностика на нестабилни етапи и последващ експериментален анализ.

3.2.2.5 Resume от checkpoint и dataloader state

Възстановяването зарежда параметри, optimizer state и dataloader прорпес (stage_index, stage_step_total, stage_states), така че обучението да продължи от последната консистентна точка.

```
@jax.jit
def _run_chunk(params, opt_state, batch_chunk, start_step, accum_grads, accum_count):
    # Една компилирана единица работа: loss/grad + accumulation + update
    (params, opt_state, _, accum_grads, accum_count), losses = jax.lax.scan(
        body, (params, opt_state, start_step, accum_grads, accum_count), batch_chunk
    )
    return params, opt_state, accum_grads, accum_count, losses
```

3.2.3 DataLoaderManager

ShardedArrowDataset + StageDataLoader реализират shard-aware четене, shuffle, batching и state restore.

```
dataset = ShardedArrowDataset(data_path)
loader = StageDataLoader(
```

```

dataset,
batch_size=batch_size,
seq_len=stage.seq_len,
shuffle=stage.shuffle,
seed=seed,
pad_token_id=pad_token_id,
)

```

3.2.4 CheckpointManager

checkpoint_manager.py капсулира запис/зареждане на параметри, optimizer state и metadata.

```

ckpt_file = save_ckpt(params, global_step, params_dir, train_loss=last_loss)
save_opt_state(opt_state, global_step, training_states_dir)
save_data_loader_state(state_path, loader_state)

```

```

# При resume
params, global_step = load_ckpt(ckpt_path)
opt_bytes = load_opt_state(global_step, training_states_dir)

```

3.2.4.1 Метод SaveParameters

Запис на .npz checkpoint с атомарно tmp -> rename поведение.

3.2.4.2 Метод SaveOptimizerState

Паралелен запис на optimizer буфери (msgpack) за коректен resume.

3.2.4.3 Метод RestoreLatest

Намиране на последната достъпна checkpoint стъпка и зареждане на съответното състояние.

3.2.4.4 Метод SetMetadata

Запис на метаданни за quality показатели (val_loss, val_ppl, train_loss).

3.2.4.5 Метод AsyncMiniCheckpoint

Мини checkpoint механизмът е част от логиката в 3.2.2.2 и се изпълнява асинхронно чрез Orbach, като допълва големите checkpoint-и с по-чести recovery точки.

3.2.5 InferenceManager

Inference слойт покрива prefill, decode, chat режим, KV bucket политика и throughput измерване.

Отбелязване: този раздел описва inference пътя в **GIANT**; при **TiDAR** prefill/decode логиката е съществено различна и е разглеждана отделно в по-късните секции.

```

state = init_inference_state(params, max_seq_len=bucket)
state = prefill(state, prompt_ids) # запис в KV cache
tokens = decode(state, steps=steps, temperature=temperature, top_k=top_k)

# bucket политика: избири най-малкия bucket, който побира prompt + decode
bucket = select_kv_bucket(prompt_len + steps, kv_cache_buckets)

```

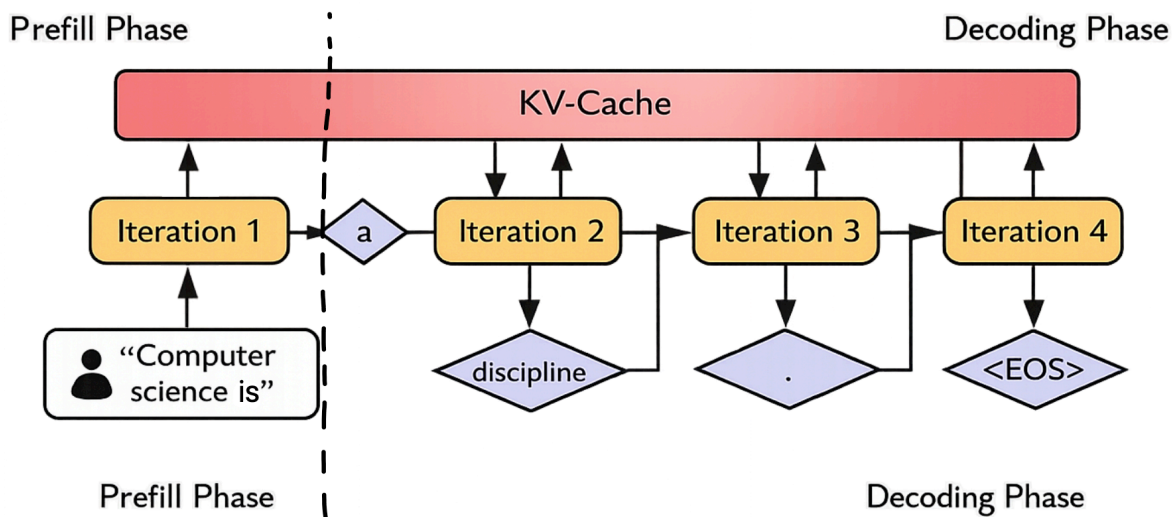


Figure 6: GIANT inference път: prefill и decode с KV cache

3.2.5.1 Метод Prefill

`jit_inference.prefill` запълва KV cache от prompt и връща начално състояние за decode.

3.2.5.2 Метод Decode

`jit_inference.decode` използва `lax.scan` за генериране на нови токени със `sampling` или `greedy` стратегия.

3.2.5.3 Метод KVBucketSelect

`Generate_faster.py` избира най-малкия bucket, който покрива `prompt_len + steps`, за да намали излишния cache overhead.

```

128
###.....
###.....
###.....

256
#####.....
#####.....
#####.....

512
#####.....
#####.....
#####.....

1024
#####
#####
#####

```

Listing 1: Bucket стратегия: избор на най-малък валиден размер вместо винаги 1024

Как да се чете диаграмата:

- # е реално използваният bucket обем.
- . е обемът, който се спестява, ако не се използва винаги 1024.

Така engine-ът избира най-малкия валиден bucket (напр. 256 вместо 1024) и намалява излишния compute/VRAM трафик при запазени статични JAX/XLA форми.

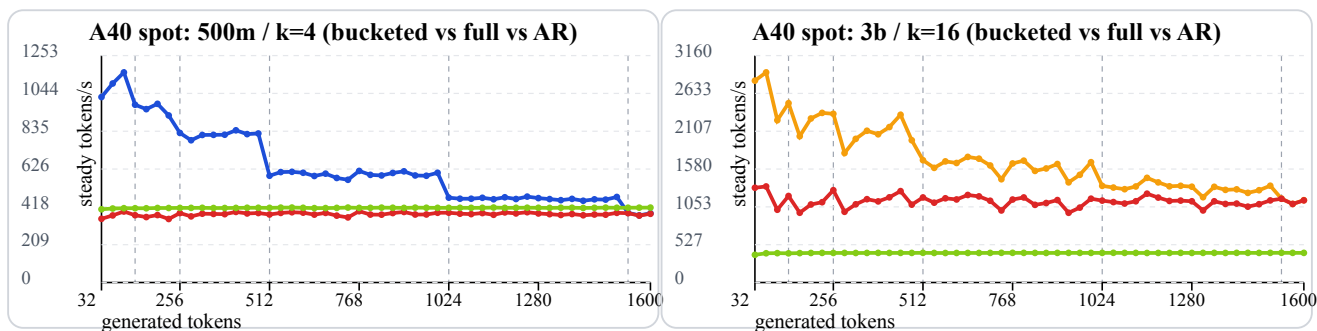


Figure 7: A40 throughput криви: 500m/k=4 и 3b/k=16

Легенда на линиите:

- синя линия — 500m bucketed (най-горна в лявата графика в повечето точки);
- червена линия — TiDAR full-context baseline (средна в лявата графика);
- зелена линия — AR baseline (най-долна в лявата графика);
- оранжева линия — 3b bucketed (най-горна в дясната графика в повечето точки).

Метриката е steady decode tokens/s (**high is better**).

В тези диаграми sweep-ът е с генерация от 0 до 1600 токена. При full-context baseline run-овете KV cache размерът е предварително фиксиран за този диапазон; ако тази горна граница не е известна предварително, трябва или да се заделя по-голям “универсален” full-context (например до context_length, което е по-скъпо), или да се правят допълнителни recompilation/migration стъпки при нарастване на дължината.

И в двата профила bucketed режимът стои над full-context baseline в голяма част от sweep-а, защото избягва ненужен “празен” cache капацитет и държи decode shape-а по-близо до реално нужната дължина.

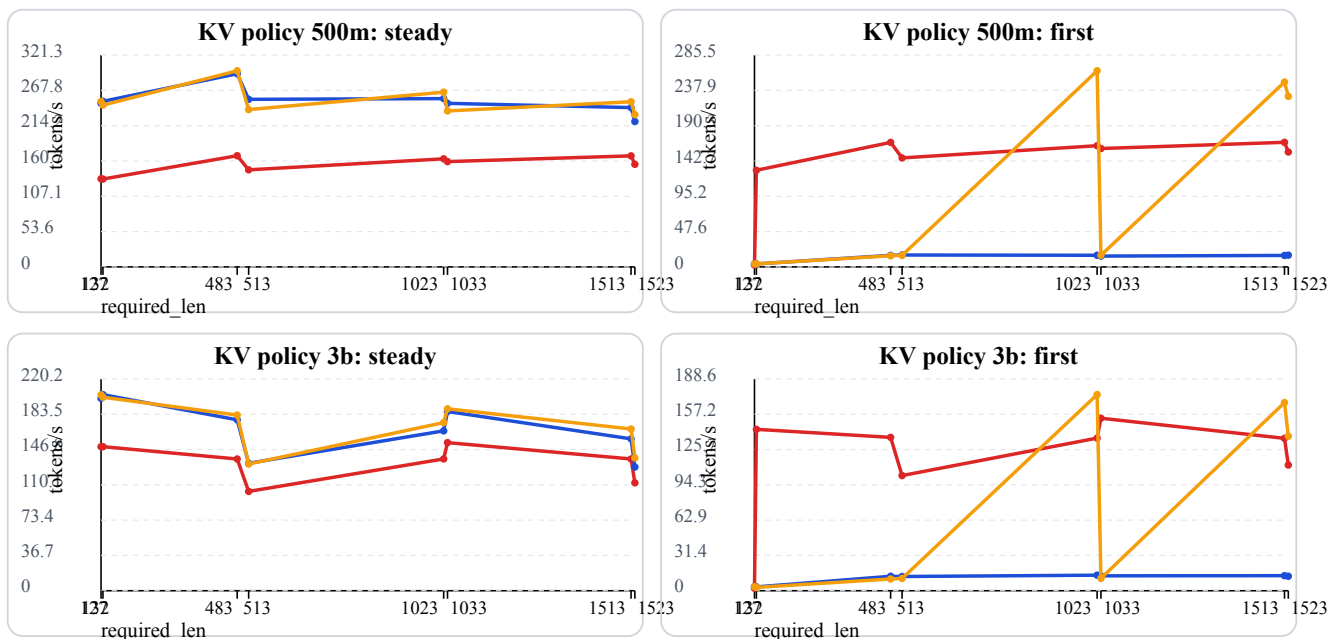


Figure 8: KV cache policy резултати (2x2) - 500m/3b, steady/first

Легенда (KV policy графики):

- червена линия — full context policy;
- синя линия — current exact (required_len);
- оранжева линия — bucketed policy.

3.2.5.4 Метод ChatTurn

Chat.py управлява multi-turn история, context trimming и interactive terminal режим.

```
python GIANT/v2/model/Chat.py \  
--checkpoint giant-data/GIANT/v2/checkpoints/params/step_0010449.npz \  
--temperature 0.8 \  
--top_k 40 \  
--steps 128
```

```
Loaded checkpoint: .../step_0121566.npz  
Chat mode: interactive (type /exit to quit)
```

```
User> Explain what attention does in one sentence.
```

```
Assistant> attention picks important old words for next token. maybe like focus.
```

```
User> And why KV cache helps?
```

```
Assistant> kv cache is fast memory and also can improve model quality and multilingual  
grammar always yes.
```

3.2.5.5 Метод BatchedPrefill

test_batched_inference.py валидира batched prefill с отмествания и проверява съвпадение с per-sample baseline.

3.2.5.6 Метод StopOnEos

Генерацията може да прекъсва на EOS токен за контрол на отговора и избягване на нежелано продължение.

3.2.5.7 Метод ThroughputReport

Отчетите измерват prefill/decode време, tokens/s и policy сравнения по контекст/модел/драфт параметри.

3.2.6 RemoteOpsManager

CICD/tools/ и Docker setup покриват deployment и remote execution. Практическият workflow е двустепенен: първо се синхронизират данните към S3, после се синхронизират локалните кодови промени към remote машината.

```
# Примерен remote workflow  
# 1) sync данни/артефакти към S3  
python CICD/tools/s3.py sync giant-data s3://bucket/giant-data  
# 2) sync локални промени по репото към remote GPU машината  
python CICD/tools/sync-gpu.py
```

3.2.6.1 Основен принцип на работа

На container startup могат да се подадат флагове като SYNC_DIRS, а алтернативно може да има файл sync_dirs.txt в root-a на S3 bucket-a. Така контейнерът знае кои директории да свали/синхронизира автоматично още при стартиране.

След това sync-gpu.py изпраща локалните (вкл. непушнати/несомит-нати) промени по кода към remote репо копието. Remote машината стартира от последния commit-нат код, а sync-gpu.py донася текущите локални редакции от работната сесия.

3.2.6.2 Кодова синхронизация преди run

Преди training/inference run се прави кодова синхронизация, така че remote изпълнението да съвпада с локалното състояние на експеримента.

Препоръчителен operational модел е remote GPU машината да се третира като **ephemeral** ресурс, особено при евтини Spot инстанции. Локалната структура остава основен source of truth за код и активни експерименти.

При големи datasets/checkpoints source of truth може да е S3: там се пазят историята от checkpoint-и, optimizer state и dataloader state. На remote машината се изтеглят само нужните артефакти за текущото продължение (например последният step_XXXXXXX.npz и съответното training state).

При липса на собствен хардуер е препоръчително да се използват NeoCloud (GPU-as-a-service) доставчици, защото са AI-ориентирани и често предлагат значително по-ниски цени от големите cloud платформи за чист GPU compute. Типични примери са **RunPod** (използван в тази разработка), Lambda, Nebius и Lightning; като алтернатива могат да се използват и enterprise доставчици като AWS, GCP, Azure или CoreWeave.

Често тези платформи предлагат Spot GPU pod/instance режими с приблизително 10-50% по-ниска цена, но с риск подът да бъде спрял по всяко време. Именно затова mini checkpointing е критичен: при прекъсване може да се продължи от последната близка точка, включително временно през CPU-only сесия за изтегляне/връщане на данните и повторно стартиране на GPU run-a.

3.3 Реализация на data pipeline

3.3.1 Документ като източник на токени

Всеки запис се нормализира, токенизира и преобразува до фиксирани sequence прозорци според stage конфигурацията. Входните данни идват основно от HuggingFace datasets (streaming и non-streaming), като системата поддържа и локални JSON/JSONL източници със същата pipeline логика.

Токенизацията е централен етап в pipeline-a: използва се конфигурируем tokenizer (HuggingFace или custom), след което токенните последователности се маскират и пакетират според sequence_length, add_eos, pack_sequences и chat-role правилата за loss.

```
# Обобщена схема на data processing конфигурацията (по build_corpus.py)
@dataclass
class StageSourceCfg:
    type: str = "huggingface"          # huggingface | json_dir
    dataset_name: str | None = None    # HF dataset
    dataset_config: str | None = None
    split: str = "train"
    streaming: bool = True
    data_files: Any | None = None

    # Текстово извличане
    text_field: str | None = None
    text_fields: list[str] = field(default_factory=list)
    join_fields: list[str] = field(default_factory=list)
    join_separator: str = " \n"
    text_template: str | None = None

    # JSON/JSONL директории
    json_root: str | None = None
    file_glob: str = "**/*.json*"

    # Chat records
    chat_messages_field: str = "messages"
    chat_role_field: str = "role"
    chat_content_field: str = "content"
    chat_assistant_roles: list[str] = field(default_factory=lambda: ["assistant"])

@dataclass
class StageCfg:
    name: str
    output_dir: str
    sequence_length: int
    target_tokens: int | None = None
    min_tokens: int = 0
```



```

pack_sequences: bool = True
add_eos: bool = True

# Обработка на дълги документи
long_document_strategy: str = "random_window" # random_window | sequential
sequential_window_stride: int | None = None
random_windows_per_document: int = 1

# Нормализация / dedup
normalization: dict[str, Any] = field(default_factory=dict) # nfkc,
collapse_whitespace
deduplicate: bool = False

# Изход
rows_per_shard: int | None = None
sources: list[StageSourceCfg] = field(default_factory=list)

```

3.3.2 Преобразуване от документ към sequence прозорци

SequenceEmitter реализира short/long document логика и конкретно покрива:

- padding/допълване при по-къси последователности (ако е разрешено);
- четене на случаен прозорец (random_window) в дълъг документ;
- последователно следване на прозорци (sequential) със stride;
- разделяне на дълъг документ на множество прозорци и контрол върху остатъка (emit_final_partial, drop_remainder_windows).

Така една и съща входна колекция може да се обработва в различни режими според целта на конкретния stage (плътно пакетирание, по-стабилно sequential покритие или по-стохастичен random sampling).

3.3.3 Директно четене и минимизиране на IO overhead

Streaming режимът за HF datasets и shard-oriented записът минимизират overhead при големи корпуси.

3.3.4 Шардване

Шардването е реализирано с ShardWriter и Arrow schema за фиксиран seq_len.

3.3.4.1 Формат на шардовете

input_ids, length, loss_mask се записват в .arrow файлове с manifest по stage.

3.3.4.2 Manifest и статистики

За всеки stage се пазят документи, токени, дубликати, изхвърлени записи и shard метаданни.

3.3.4.3 Контрол на паметта

Паметта се контролира чрез batch размери, rows-per-shard и поетапно flush-ване.

3.3.5 Loader batch изграждане

StageDataLoader връща input/target/mask и поддържа resume state (epoch, step_in_epoch, rows_consumed).

3.3.5.1 Prefetch като оптимизация

_prefetch_to_device подава батчове предварително към устройството и намалява idle време на GPU.

3.3.6 Допълнителни preprocessing опции

Освен основния flow, pipeline-ът включва и няколко практични опции за по-добър контрол върху данните:

- assistant-aware chat маски, така че loss да се натрупва само върху целевите роли;
- NFKC и whitespace normalization по stage;
- опционална hash-базирана дедупликация;

- JSON/JSONL ingestion с line-wise/streaming четене;
- атомарен запис и stage-by-stage преизпълнение за устойчивост при прекъсвания.

3.4 Имплементация на “Think in Diffusion, Talk in AutoRegression”

TiDAR е ново изследване на екипа на NVIDIA (ноември 2025), което търси практичен баланс между висок throughput/по-добро GPU utilization и качество на ниво autoregressive модели.

Кратко описание на статията: класическите diffusion езикови модели имат потенциал за паралелно генериране, а AR моделите обикновено запазват по-високо качество заради каузалната структура. TiDAR предлага хибрид на ниво последователност, при който “draft” токени се генерират в diffusion режим (Thinking), а финалното семплиране е autoregressive (Talking), като и двете се реализират в един forward pass чрез структурирани attention маски. Според публикуваните резултати архитектурата е serving-friendly, поддържа точен KV cache и показва по-висок throughput спрямо speculative decoding и предишни diffusion варианти, като за първи път затваря качествена дупка до AR при съществено по-висока скорост (порядък 4.71x-5.91x tokens/s по данни на NVIDIA).

3.4.0.1 Контекст: от класически speculative decode към TiDAR

Преди TiDAR, най-честият practical подход за ускорение е класически speculative decoding с draft + verify логика. Този подход е добра отправна точка, но често страда или от по-слаб draft модел, или от по-ниска ефективност на верификацията.

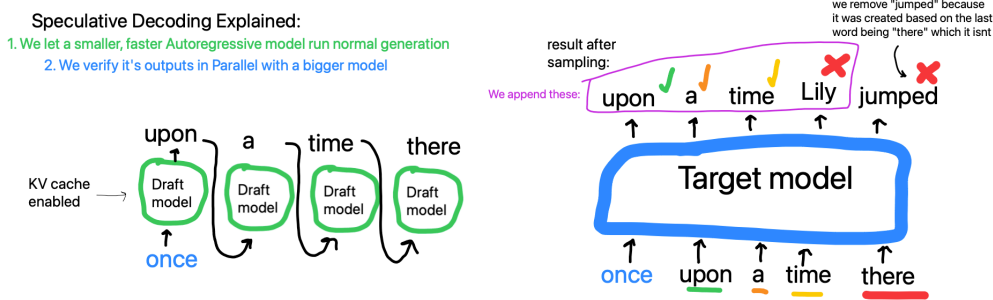


Figure 9: Базов speculative decoding (референтен случай)

В класическия вариант малък draft модел q и голям verify модел p работят с един и същ tokenizer и една и съща токена азбука. Draft моделът предлага последователност от кандидати $x_1, \dots, x_K$, а verify моделът ги проверява каузално за същия префикс.

$$\alpha_i = \min\left(1, \frac{p_i(x_i)}{q_i(x_i)}\right)$$

Тук α_i е вероятността за приемане на токена x_i на позиция i ; ако токена се отхвърли, се семплира от коригираща дистрибуция:

$$r_{i(v)} = \frac{\max(0, p_{i(v)} - q_{i(v)})}{Z_i}$$

За KV cache е важно speculative токени да не се commit-ват окончателно предварително. Записва се само приетият префикс (или се прави rollback до приетата дължина), иначе cache състоянието се разминава с валидирания контекст и decode-ът деградира.

3.4.0.2 Основна идея на TiDAR в един forward pass

TiDAR променя тази схема, като комбинира verify + predraft в един structured forward pass. Така се използва по-добре наличният паралелен compute и се намалява serving overhead-ът.

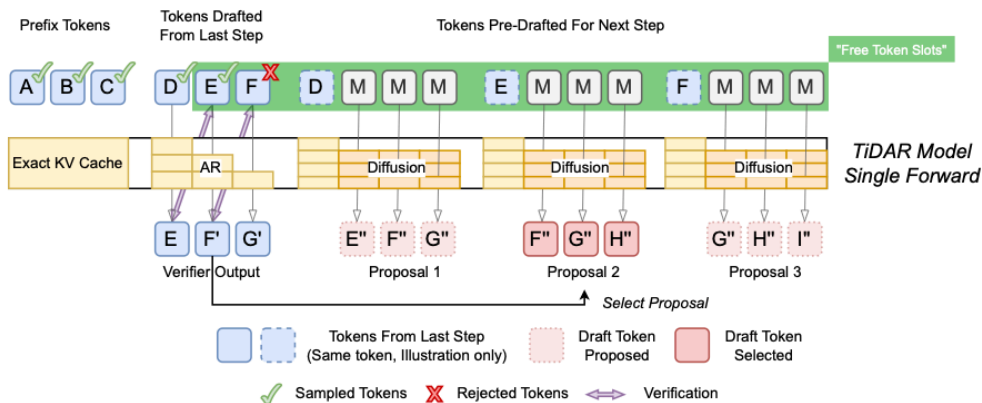


Figure 10: TiDAR single-forward layout: verify + predraft (diagram from TiDAR paper)

На схемата verify частта оценява текущия draft, а predraft частта едновременно предлага следващите кандидати за идната итерация. Ключовият момент е, че това не са два отделни модела, а един и същ TiDAR модел, изпълнен веднъж с различно структурирани attention връзки в рамките на същия forward pass.

Така се спестява допълнителен model orchestration overhead (няма втори draft модел и допълнителен cross-model sync), а наличният GPU compute се използва по-плътно в една обща граф операция.

3.4.0.3 Структурирани маски и достъп по позиции

Ключът е в маските: различни части от входа имат различен режим на внимание, така че едновременно да се пази AR логиката за “talk” и diffusion паралелизмът за “think”.

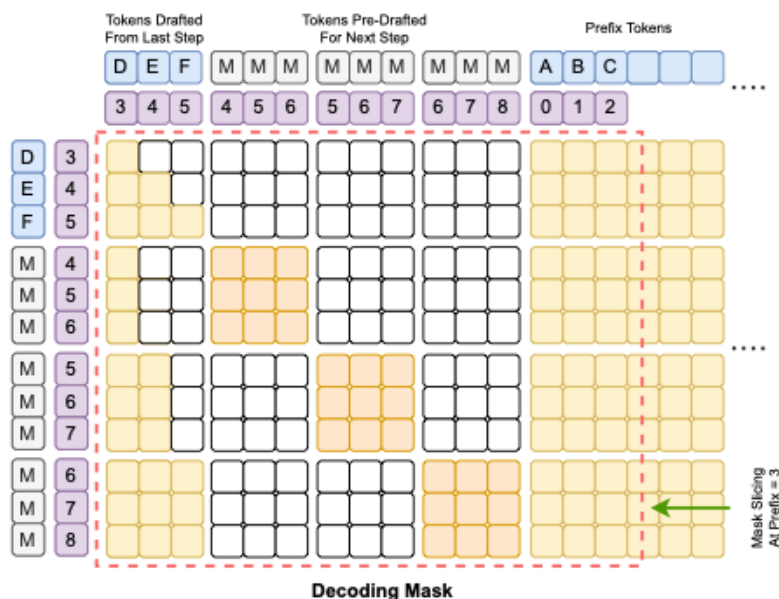


Figure 11: Структурирана маска за TiDAR decode/преддрафт логика (diagram from TiDAR paper)

Verify tokenите виждат каузален контекст по същия принцип както при нормален speculative decoding. Predraft tokenите виждат останалите токени в своя predraft group с bidirectional attention и едновременно с това виждат префикса до позицията, след която трябва да предсказват, т.е. до съответния draft token.

3.4.0.4 Prefill маска

Prefill фазата подготвя началното KV-cache състояние за decode, като подава prompt контекста с коректна каузална видимост. Това гарантира, че последващите verify/predraft стъпки стъпват върху консистентна базова история.

От prefill изхода се взима и първият D* token (първият draft token), който служи като стартова точка за следващата decode итерация.

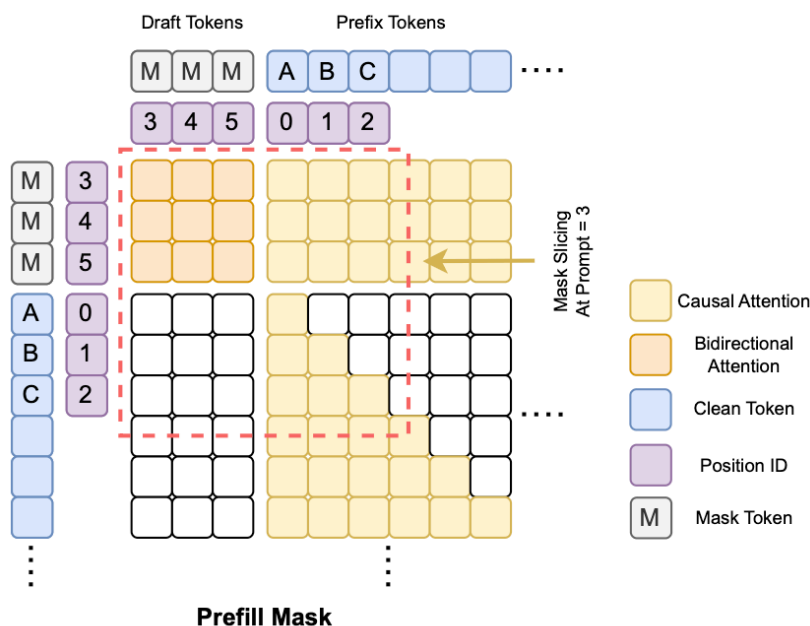


Figure 12: TiDAR prefill маска (diagram from TiDAR paper)

3.4.0.5 Training маска

В training режим маската реализира едновременно AR и diffusion режим върху подравнени позиции, така че моделът да учи и следващ token (causal), и паралелна denoising логика в diffusion частта.

Критичното предимство е, че training входът е с удвоен контекст [clean | diff] (приблизително 2S), а не с decode-подобна експлозия от типа $K + K^2$. Така TiDAR може да се тренира с по-управляем memory/compute профил и със стандартен training pipeline за фиксирана контекстна дължина.

В paper-а е тествано corruption поведение с шум, маска и смесени режими; отчетено е, че вариантът само с mask corruption работи най-добре. Затова и в тази имплементация се използва единен [MASK] token при diffusion частта на training/inference подредбата.

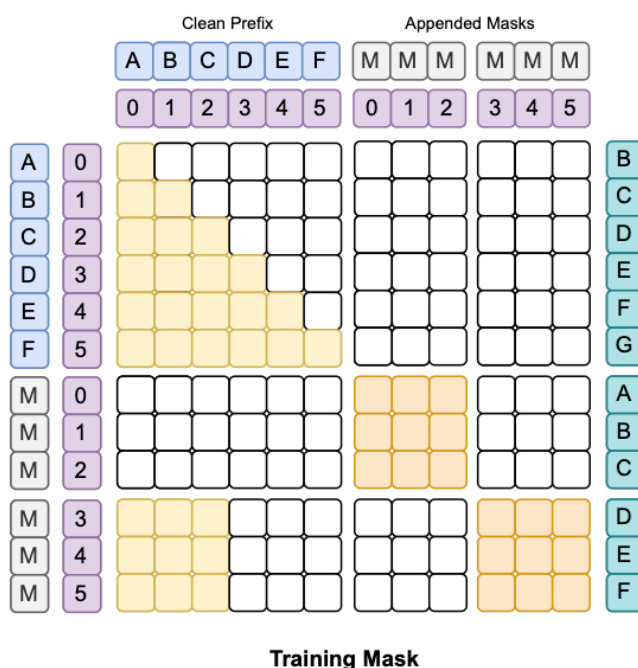


Figure 13: TiDAR training маска (diagram from TiDAR paper)

3.4.0.6 Loss формулировка и баланс AR:Diff

В оригиналната формулировка training целта е:

$$L_{T(\omega)} = \frac{1}{1 + \epsilon} \left(\frac{\epsilon}{S} \sum_{i=1}^S L_1(x_i, x_{i+1}; \omega) + \frac{1}{S} \sum_{i=1}^S L_2(m, x_i; \omega) \right)$$

където L_1 е AR терминът, L_2 е diffusion терминът, m е [MASK] токена, а ϵ контролира баланса между двата термина.

В paper-а е направен sweep за съотношението между двата термина и е наблюдавано, че баланс 1:1 (т.е. $\alpha = \beta = 1$) е най-стабилен и дава най-добър общ компромис. Тестваният диапазон е около 0.8 .. 1.2 за всеки коефициент.

В тази работа се приема същият принцип: AR и Diff компонентите се държат балансирани като базова настройка, а отклоненията се използват само при целеви експерименти.

3.4.0.7 Anchor-TiDAR (финален акцент)

Anchor разширението въвежда стабилна референтна точка в decode стъпката и подобрява практическата ефективност на приемане/rollback логиката, особено при дълги генерации.

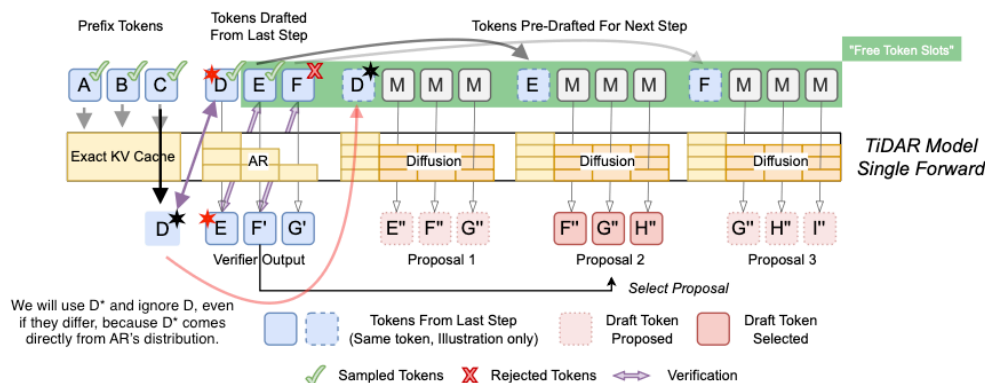


Figure 14: Anchor-TiDAR forward pass

Важно уточнение: Anchor-TiDAR е мое собствено разширение спрямо оригиналния TiDAR paper. В статията акцентът е по-скоро върху това моделът да е обучен достатъчно добре, така че пълен rejection (нула приети draft токени) да се случва минимално рядко, вместо да се описва изрично fallback механизъм за този случай. Ако все пак се случи full rejection, трябва или да се направи нов predraft pass, или да се държи допълнителен K набор (още compute), за да се избегне стоп в прогреса.

С Anchor подобрението се гарантира прогрес без нужда от допълнителен predraft compute в тази гранична ситуация, като целта е да се запази качеството на AR изхода. По-нататък е обяснено и как се управлява KV-cache pool-ът със static share политика.

3.4.1 Представяне на dual sequence вход

TiDAR training конструира вход с дължина 2S във формат [clean | diff]:

- clean част (1..S) — стандартен AR поток за next-token предсказване;
- diff част (S+1..2S) — diffusion поток с [MASK] corruption и structured visibility.

На практика се учат две съгласувани задачи върху едни и същи таргети:

- AR: $x_i \rightarrow x_{(i+1)}$ по каузален ред;
- Diff: $\text{masked}(x_i) \rightarrow x_i$ в паралелен blockwise режим.

Това подравняване позволява директно сравнение и комбиниране на AR/Diff логити по позиции, което после се използва и при verify/predraft decode логиката.

Визуална референция за training маската: Figure 13.

3.4.2 Преобразуване и маскиране по позиции

`tidar_masks.build_tidar_train_bias` създава attention bias матрица, която управлява точно кои позиции се виждат в training pass-a. Ключовите правила са:

- clean -> clean: каузален достъп (класически AR);
- diff вътре в block: bidirectional достъп;
- diff -> clean prefix: разрешен достъп до нужния контекст;
- clean/diff към бъдещи невалидни позиции: забранен достъп.

Тези правила гарантират, че AR частта остава каузално коректна, а diffusion частта получава паралелна локална видимост без leakage към бъдеща информация извън допустимия контекст.

Decode/предрафт маската е показана по-горе на Figure 11.

3.4.3 Извличане на verify и predraft логити

При inference входът се подрежда като `[current_draft | predraft_masks]`, след което от един forward pass се извличат:

- verify логити за текущия draft;
- K predraft предложения за следващата стъпка.

Технически това е важно, защото verify и predraft не се изпълняват като две отделни model извиквания. Логитите се взимат от различни позиционни срезове на един и същ output тензор, което:

- намалява host orchestration overhead;
- подобрява ефективността при static-shape изпълнение;
- поддържа консистентен KV-cache update модел за следващата итерация.

3.4.4 KV-cache pointer commit/rollback

Anchor-TiDAR използва optimistic KV write и pointer semantics: приеманата част се commit-ва чрез `prefix_len`, а отхвърленият суфикс се "rollback-ва" логически чрез връщане на pointer-a.

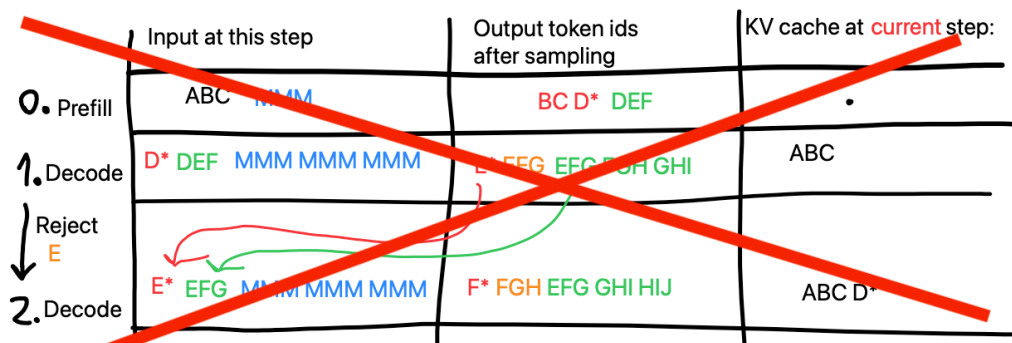


Figure 15: KV cache развитие по итерации при Anchor-TiDAR

3.4.5 Decode сценарий (Anchor-TiDAR)

- 1) Prefill: записваме prompt в KV cache и взимаме стартов D*.
- 2) Forward pass: едновременно получаваме verify логити + K predraft предложения.
- 3) Acceptance: проверяваме предложените токени по ред и приемаме най-дългия валиден префикс.
- 4) Commit/Rollback: KV pointer се премества само до приетата дължина.
- 5) Next step: от приетия префикс + новия anchor продължаваме следващата итерация.

Визуални референции: Figure 14 и Figure 10.

3.4.6 Допълнителна визуална интерпретация за Mij

Нека A е anchor token, D1..Dk са draft токени, а Mij е token в predraft блок i на позиция j.

Mij вижда:

- каузално: префикса + anchor + нужния verify префикс;
- bidirectional: токени в собствения си predraft блок;
- не вижда: нерелевантните бъдещи блокове извън текущата structured група.

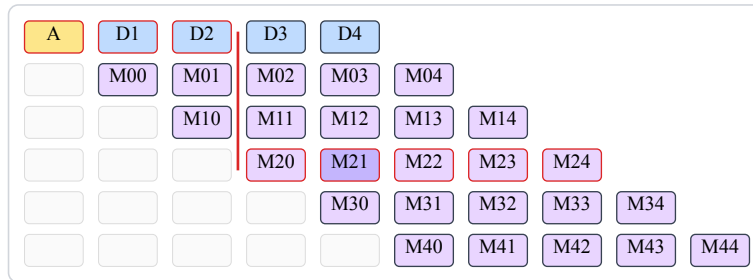


Figure 16: Triangular visualize на verify/predraft зависимостите; пример с маркиран M21

При token M21:

- каузално внимание: prefix + A + D1 + D2;
- двупосочно внимание: M20...M24 (в рамките на собствения блок);
- липса на внимание към останалите предрафт блокове.

Сравнение с класически speculative decode: Figure 9.

3.5 Експеримент: Free Token Slots

В този раздел се анализира явлението “Free Token Slots” - случаи, в които TiDAR може да използва допълнителна паралелна работа в една decode стъпка по-ефективно от класически AR decode път. Експериментите са върху A40 профили и са свързани директно с избора на structured masks, single-forward verify/predraft и KV-cache стратегията, описани по-горе.

В TiDAR paper-а има сходен тип анализ, но тук той е повторен върху значително по-слаб хардуер (A40 вместо H100), като е разширен и с допълнителни тестове за sampling режими и greedy decoding поведение.

Точно това явление е и основната мотивация зад начина, по който TiDAR е конструиран: да запълва “свободните” изчислителни слотове в decode стъпката с полезна verify/predraft работа, вместо GPU ресурсът да остава неизползван между последователните AR операции.

Важно за интерпретацията: показаните throughput резултати са benchmark-нати при зададен теоретичен accept_rate = 0.8 (около 80% приети токени на итерация). За тези експериментални матрици не са тренирани отделни нови модели за всеки сценарий; сравняват се скоростни профили при фиксирана теоретична accuracy настройка.

3.5.1 Native decode attention

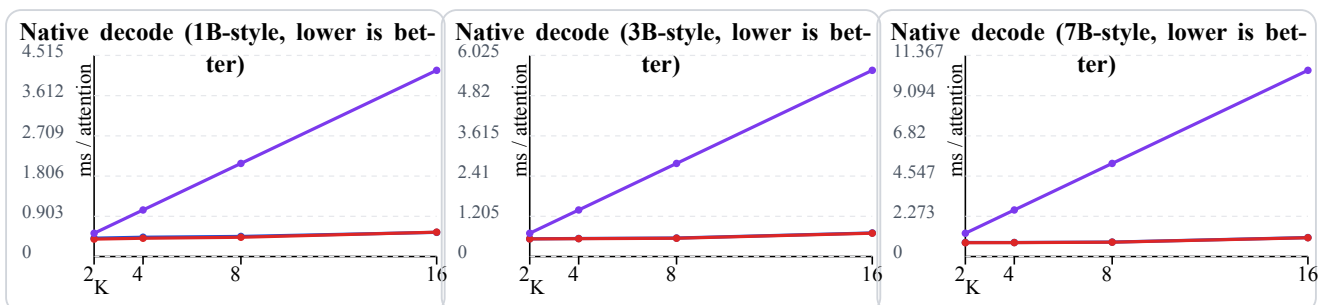


Figure 17: Free Token Slots - native decode attention при 1B, 3B, 7B (A40)

Легенда (decode attention графики):

- ■ синя линия — TiDAR native structured (стандартният decode път със structured mask; почти припокрива се с червената);

- червена линия — TiDAR native dense / no-mask variant (същият decode път, но без подаване на structured mask; долната от двете близки TiDAR линии);
- лилава линия — AR baseline ($K \times \text{len1}$, горната линия).

Наблюдението тук е, че TiDAR native decode кривите остават значително по-плоски спрямо AR baseline, който е мащабиран като $K \times \text{len1}$. Това е practically важният сигнал за проекта: при нарастване на K се получава по-добра амортизация на compute разхода на стъпка и по-добро използване на наличния паралелен ресурс.

3.5.2 Kernel terms (контролен анализ)

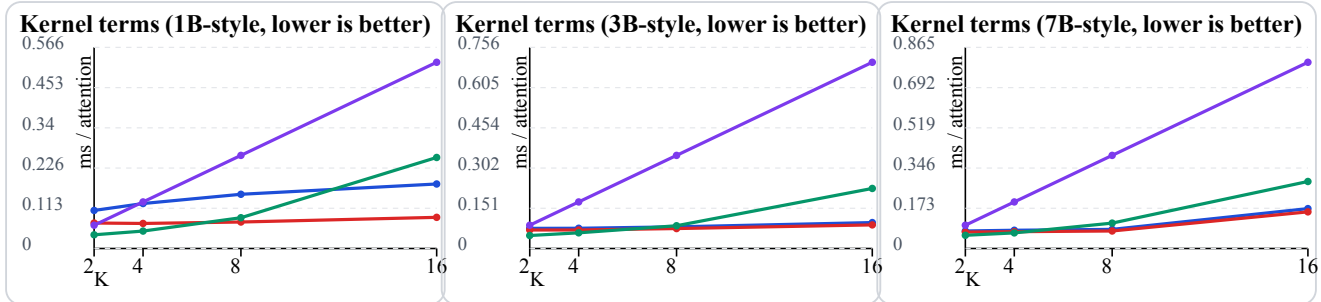


Figure 18: Kernel terms експеримент за attention pass варианти (1B, 3B, 7B)

Легенда (kernel terms графики):

- синя линия — kernel structured (реално използваният вариант);
- червена линия — kernel dense / no-mask (почти винаги най-долна);
- зелена линия — dense + jax-zero variant (при $K=2,4$ е най-долно, при $K>4$ започва да расте);
- лилава линия — AR baseline ($K \times \text{len1}$, consistently най-горна права линия).

Тези графики са контролен експеримент за различни attention pass варианти и целят да изолират ефекта на маската върху compute профила. Важно: тук не се влиза в custom kernel посока (Pallas/Triton/собствени CUDA/C++ kernel-и); анализът е на текущия production-like decode път. Както и в paper-a, custom kernel оптимизациите остават по-скоро посока за бъдещо развитие.

3.5.3 MLP scaling

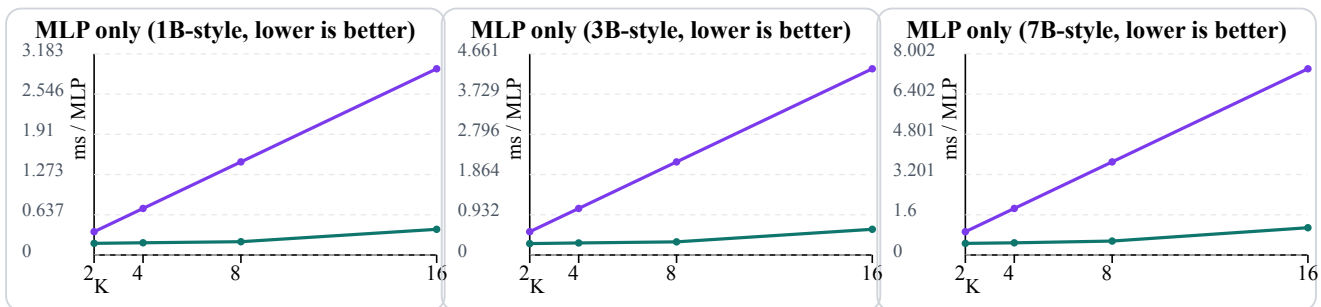


Figure 19: Free Token Slots - MLP scaling сравнение (1B, 3B, 7B)

Легенда (MLP графики):

- тюркоазена линия — TiDAR MLP при $L = K + K^2$ (долната линия);
- лилава линия — AR MLP baseline ($K \times \text{len1}$, горната линия).

MLP графиките подсилват същата идея: при TiDAR токеният блок $L = K + K^2$ се обработва в един общ pass, докато AR baseline се акумулира почти линейно с K . За текущата архитектура това е ключова връзка между теорията и практиката - structured single-model decode пътят не само работи коректно, а и носи реална throughput полза в настройките, върху които е разработен проектът.

3.5.4 Sampling path (greedy и non-greedy)

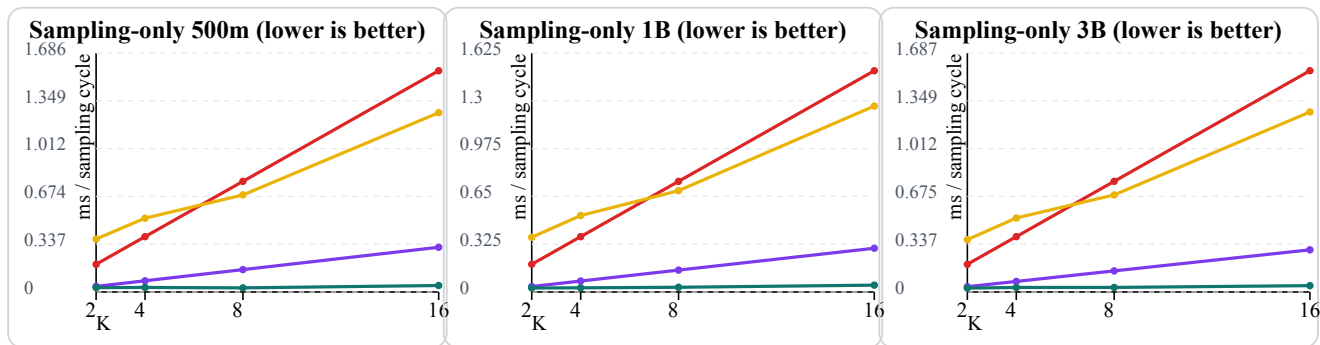


Figure 20: Free Token Slots - sampling-only сравнение (500m, 1B, 3B)

Легенда (sampling графики):

- тюркоазена линия — TiDAR staged sampling, greedy ($\text{top}_k=0$);
- лилава линия — AR scaled baseline, greedy ($K \times \text{len1}$);
- жълта линия — TiDAR staged sampling, non-greedy ($\text{top}_k=50$);
- червена линия — AR scaled baseline, non-greedy ($\text{top}_k=50$).

Sampling-only резултатите показват, че при greedy режим TiDAR държи по-нисък sampling cycle почти по целия диапазон на K . При non-greedy ($\text{top}_k=50$) има crossover поведение: при ниски K разликата е малка/в полза на AR, а при по-високи K TiDAR започва да печели, което е в синхрон с целта да се използва по-добре паралелният compute при по-широк draft.

Как работи staged sampling в този benchmark: първо се семплира verify токенът за текущата позиция, след което се прави rejection/select стъпка за predraft предложенията и се семплира само избраният predraft ред за следващата итерация. Така се мери изолирано именно sampling/rejection пътят (без transformer forward), което позволява директно сравнение с AR $K \times \text{len1}$ baseline.

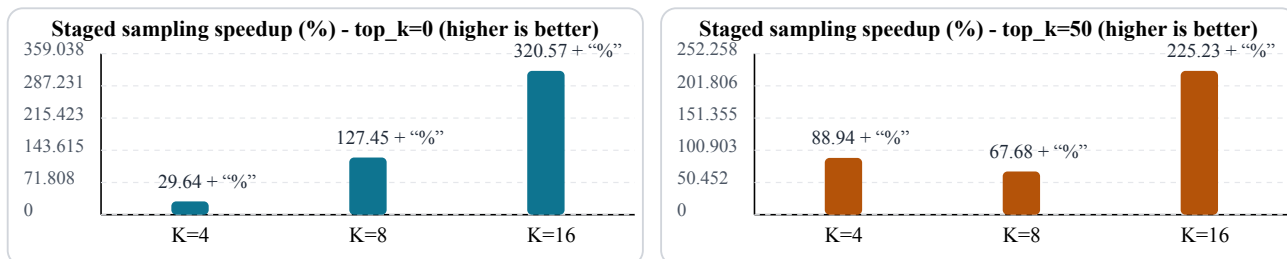


Figure 21: Изолиран sampling speedup: staged path спрямо old path

Практически извод: в тези run-ове sampling частта е малък дял от целия decode wall-time (приблизително под 1%). Това се вижда от факта, че изолираният sampling speedup е голям, но end-to-end decode подобрението остава около десети от процента; следователно основният bottleneck остава transformer forward пътят, а не самото sampling действие. Въпреки това free token slots остават важни, защото показват къде има неизползван паралелен ресурс и къде следващите оптимизации могат да донесат допълнителен throughput.

3.6 Резултати: TiDAR срещу AR (A40 и H100)

След анализа на Free Token Slots тук показваме head-to-head резултатите за TiDAR срещу AR при еднаква 3b конфигурация върху A40 и H100.

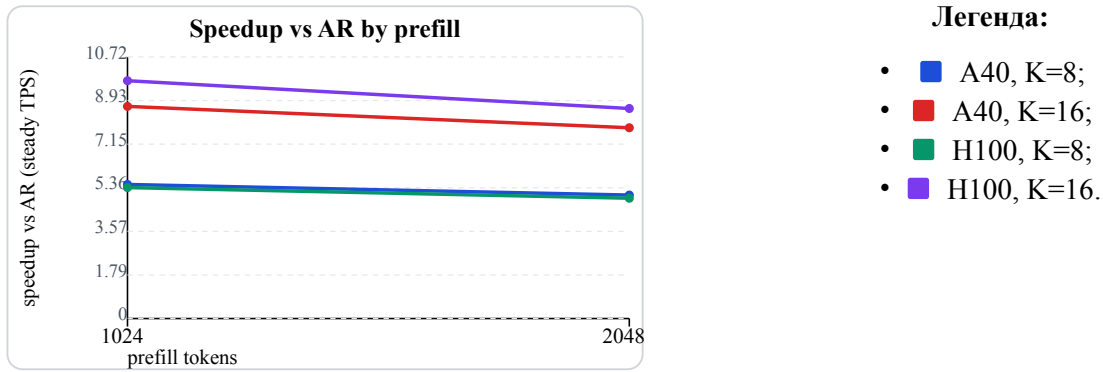


Figure 22: Head-to-head: speedup vs AR по prefill (A40 + H100)

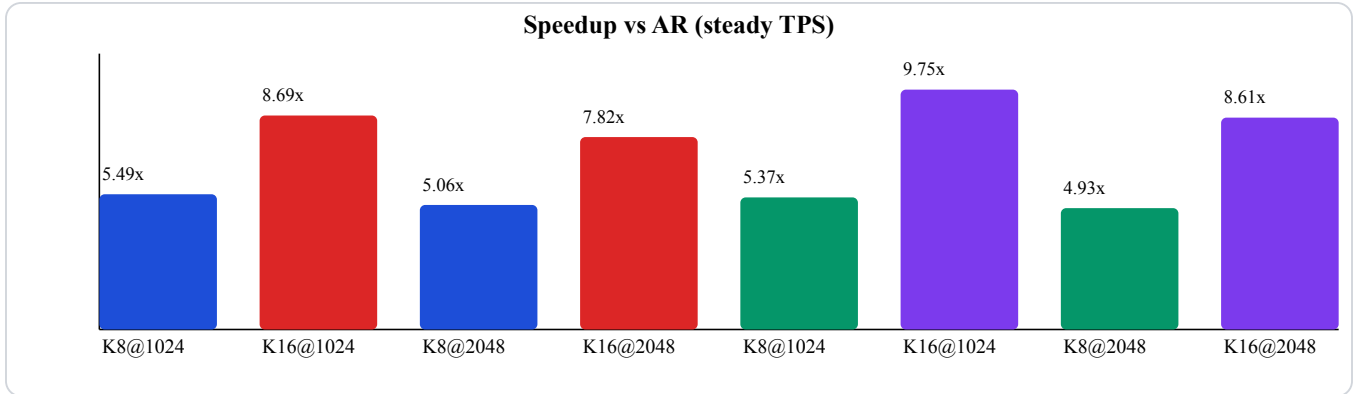


Figure 23: Head-to-head: обобщена speedup диаграма (full width)

NOTE TO SELF: Re-run A40 vs H100 head-to-head because no shot H100 to lose to A40, although its prefill and it is really short so can be random. For that run just a huge prefill test.

Тези резултати потвърждават, че наблюдаваните ускорения не са единичен артефакт от една карта, а се запазват и при по-висок клас GPU, като при K=16 H100 показва най-силна speedup крива.

3.7 Разширена loss функция за TiDAR

Тази секция описва разширената loss формулировка в имплементацията, при която отделните обучителни термини са с независими коефициенти и могат да се комбинират контролирано според целта на конкретния training stage.

3.7.1 Формулировка с независими коефициенти

В практическата имплементация замених нормализацията от $\text{ramp-}a / (1 + \alpha)$ с модулна сума от отделни термини, всеки със собствен коефициент. Така всеки компонент може да се усилва, отслабва или да се изключва с нулев коефициент, което прави възможни чисти ablation експерименти и целенасочени комбинации между различни обучителни сигнали.

$$L = \alpha L_A + \beta L_D + \rho D_f + \chi D_r + \delta L_H + \delta_m L_P + \eta L_S + \gamma L_K$$

Където $\text{bar}(P_A)$ означава AR разпределение със stop-gradient (без обратен градиент към AR клона).

- AR термин:

$$L_A = -\frac{1}{S-1} \sum_{t=0}^{S-2} \log P_{A,t}(x_{t+1})$$

Поддържа стандартната next-token езикова способност в clean половината.

- Diff термин:

$$L_D = -\frac{1}{S-1} \sum_{t=0}^{S-2} \log Q_{D,t}(x_{t+1})$$

Учи diffusion половината да реконструира същата бъдеща цел от mask-нат вход.

- Forward KL термин:

$$D_f = \sum_v |(P_A(v)) \log \frac{|(P_A(v))|}{Q_D(v)}$$

Наказва Diff, когато изпуска вероятностна маса, която AR счита за важна.

- Reverse KL термин:

$$D_r = \sum_v Q_D(v) \log \frac{Q_D(v)}{|(P_A(v))|}$$

Наказва Diff, когато разпределя излишна маса извън AR разпределението.

- Hard agreement термин:

$$L_H = -\log Q_D(v^*)$$

Притиска greedy избора на Diff да съвпада с greedy избора на AR; v^* е токeнът с най-висока вероятност според AR.

- Masked hard agreement термин (prefix mask):

$$L_P = \frac{1}{N_P} \sum_{i \in P} -\log Q_{D,i}(v_{A,i}^*)$$

Това е новият “маскиран” hard loss: взима същия hard сигнал, но само върху позициите до първото несъвпадение между AR и Diff (включително). В кода този термин се управлява с отделен коефициент `delta_masked` и служи да фокусира натиска върху префикса, който е най-важен за асерт логиката.

- Soft distillation термин:

$$L_S = T^2 D(p_A^T \parallel p_D^T)$$

Пренася “меката” форма на AR разпределението към Diff, без да се фиксира само един токeн; p^T са температурно-скалирани вероятности.

- Top-K set distillation термин:

$$L_K = -\log \sum_{v \in V_K} Q_D(v)$$

Концентрира вероятностната маса на Diff в топ-K набора на AR и подобрява шанса за асерт при decode; V_K е множеството от топ-K AR кандидати.

Важно техническо поведение в кода: при коефициент θ съответният термин се пропуска напълно от изчислението (`alpha`, `beta`, `rho`, `chi`, `delta`, `delta_masked`, `eta`, `gamma`), което позволява експериментите да се правят с минимален излишен compute и с ясна изолация на ефекта от всеки компонент.

Механизъм на JAX компилацията: в `TiDAR/model/Run_training.py` функцията `_run_chunk` е JIT-компилирана чрез `@partial(jax.jit, static_argnames=...)`, като loss коефициентите са подадени като static аргументи. При нова комбинация от стойности се компилира нов изпълним вариант, а при повторение на същата комбинация се използва кешираният вариант. Тъй като branch-овете в `TiDAR/model/Training_step.py` са условни по коефициент, изключените термини се `prune`-ват от съответния `jaxpr`.

3.8 Резултати от greedy run-ове (135M vs 360M)

Тази секция обобщава резултати от реално тренирани TiDAR варианти с `draft_len=8`: 135M и 360M. Данните са от продължителен run (около 5 часа), при който по-големият модел е стартиран с по-голям batch, а логовете са подравнени по optimizer стъпки.

Легенда за сравняваните модели:

- 135M модел - run на RTX 5090 (32GB), платена цена за наем 5.23 USD;
- 360M модел - run на RTX PRO 6000 (96GB), платена цена за наем 11.81 USD.
- сива линия = делта 135M / 360M (1.0 означава равни стойности).

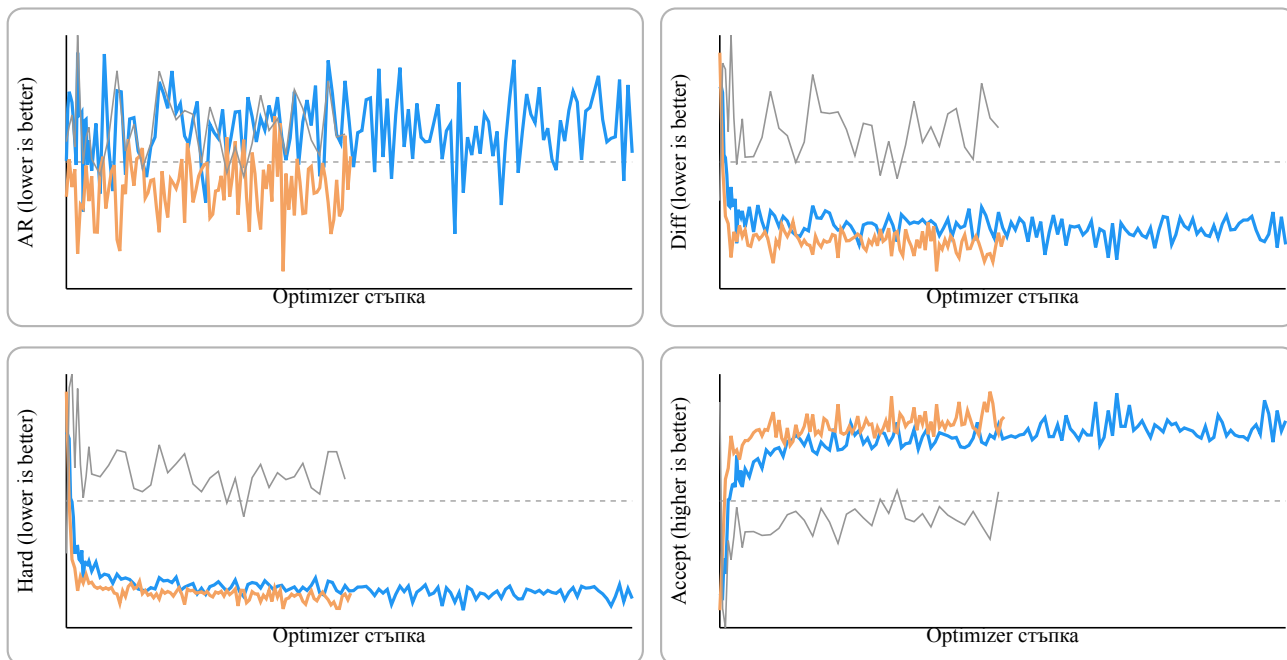


Figure 24: Обучителни метрики: 135M срещу 360M

Практически извод: 360M вариантът държи по-стабилни и по-ниски Diff/Hard загуби и по-висок greedy acceptance, но идва с осезаемо по-висока цена във време и ресурс. Моделът е приблизително 2.66 пъти по-голям по брой параметри спрямо 135M, а в реалните run-ове обучението му беше около 3-4 пъти по-бавно. Тези диаграми показват и че началните способности на базовия модел директно се отразяват върху ученето на TiDAR future prediction-ите: по-силният стартов модел учи по-стабилно тази задача, което е консистентно с по-големия му капацитет и с факта, че е предварително трениран върху повече данни. За проекта това е важен ориентир за trade-off между качество, цена и време за итерация при TiDAR training.

3.9 Проведен експеримент с различни loss функции и конфигурации

В този раздел е представен контролиран експеримент за TiDAR post-training с множество loss конфигурации. Основната цел е да се оцени кои комбинации подобряват ключовите метрики (Diffusion loss и Greedy acceptance), като едновременно с това се запази качеството на оригиналния AR модел (без значимо влошаване на AR loss спрямо последните итерации от базовото GIANT обучение). Дизайнът е избран така, че да поддържа бърз експериментален цикъл: промяна на loss конфигурация, кратък run, метрика-анализ и следваща итерация.

3.9.0 Избор на TinyStories и базова конфигурация

За експерименталната серия е избран TinyStories, защото е сравнително малък корпус с ниска ентропия. Това позволява бързи итерации¹ и ясна интерпретация на ефекта от loss промените. В практиката този тип корпус се научава стабилно и от базов модел около 30M параметъра, без да е необходимо пълно минаване през целия набор във всеки run.

Базовият GIANT модел в тази серия е дефиниран със следната конфигурация:

```

model:
  embedding_size: 384
  num_heads: 6
  num_kv_heads: 2
  num_layers: 18
  feed_forward_size: 1024
  context_length: 512
training:
  batch_size: 32
  gradient_accumulation: 4
stages:
  - dataset: tinystories_512
    seq_len: 512
    epochs: 2
    fraction: 0.75

```

След базовото AR обучение се преминава към TiDAR post-training със същата архитектура, като началният режим е $\alpha=1$, $\beta=1$:

```

tidar:
  draft_length: 6
training:
  batch_size: 16
  gradient_accumulation: 4
loss:
  alpha: 1.0
  beta: 1.0
  rho: 0.0
  chi: 0.0
  delta: 0.0
  eta: 0.0
stages:
  - dataset: tinystories_512
    seq_len: 512
    epochs: 3
    fraction: 0.75

```

Примерни изречения/промпт фрагменти от корпуса:

- “There was a small village.”
- “Once upon a time, a little girl...”
- “In the forest, a tiny fox...”
- “One day, the teacher said...”
- “The robot wanted to learn...”

¹ Тренирането на базовия GIANT 30M параметров модел върху 1.05 милиарда токена отне 33 минути на RTX 5090 (0.89 USD/час), а всеки TiDAR post-training run около 2.5 часа на същия хардуер. Целият експеримент беше проведен в продължение на седмица, защото изискваше междинен анализ след всеки run (включително логитни хистограми), и струва общо около 32 USD. 

3.9.1 Експериментален протокол и избор на checkpoint

Основният TiDAR baseline run ($\alpha=1$, $\beta=1$) достига 121566 optimizer стъпки. За сравнителната част е избран checkpoint около 90k, от който се пускат над 10 различни loss конфигурации до приблизително 121k, за да се измери ефектът им върху Diffusion/Greedy метриките при близки стартови условия.

Изборът на 90k е целенасочен: в този етап baseline режимът започва видимо да забавя нормалното си учене при $\alpha=\beta=1$, което го прави подходяща точка за branch сравнения. Част от експериментите са стартирани и от нулева стъпка за допълнителна проверка на ранната динамика.

Branch-based дизайнът е силен за сравнение, защото:

- елиминира ефекта от различна ранна инициализация;
- сравнява режимите върху близка checkpoint основа;
- позволява директни delta криви спрямо baseline по една и съща step ос.

Във файла са използвани два прозореца на анализ:

- пълен прозорец (за стабилния run) 0 - 121k за глобална динамика;
- zoom прозорец 90k - 121k за head-to-head сравнение между branch вариантите.

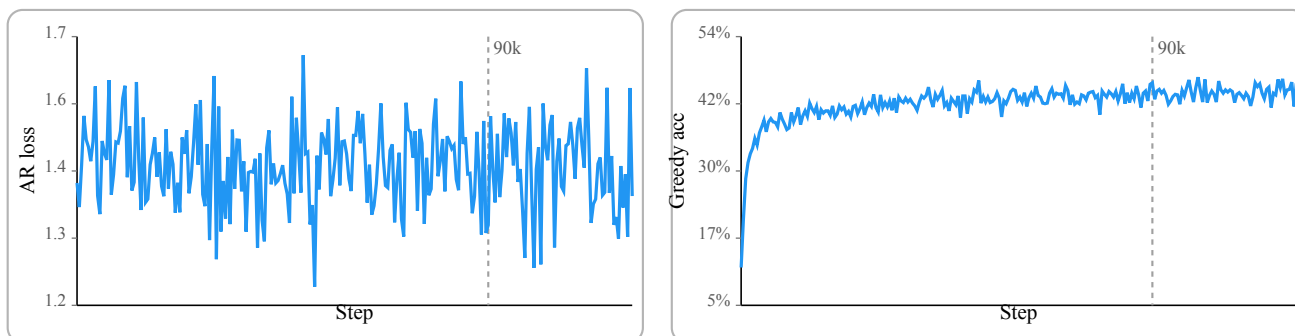


Figure 25: Базова траектория и branch cut при 90k стъпка

3.9.2 Сравнявани loss режими

Легендата по-долу обобщава сравняваните конфигурации в sweep-а. Тя покрива както “меки” alignment сигнали (KL/Distill), така и “твърди” acceptance-ориентирани сигнали (delta, delta_masked, top-k set), което е методологично правилно за TiDAR, където целта не е само нисък loss, а по-ефективен speculative decode.

Легенда на вариантите:

- ■ Stable - $\alpha=1$, $\beta=1$, референтна линия;
- ■ KL-only - $\alpha=0.2$, $\beta=0$, $\rho=0.5$, $\delta=0.3$, без Diff CE;
- ■ KL-keep - $\alpha=1$, $\beta=1$, $\rho=0.1$, $\delta=0.3$, умерен alignment;
- ■ Distill - $\alpha=1$, $\beta=1$, $\eta=0.04$, $T=2$, мека дистилация AR->Diff;
- ■ Greedy eta - агресивен agreement/дистилация режим;
- ■ Bigger beta - $\alpha=1$, $\beta=5$, усилен Diff натиск;
- ■ Top-K set - $\gamma=0.01$, $\gamma_{\text{topk}}=8$, късен stage;
- ■ Big Gamma+Delta - силен Top-K + hard agreement;
- ■ Masked Delta later - частичен 90k+ run с δ_{masked} ;
- ■ Delta masked early - кратък ранен run под 30k.

Run-ове, които водят до твърде бърза деградация на AR quality (catastrophic forgetting) или показват незначим ефект спрямо основната група, не са включени във финалното сравнение в тази секция.

3.9.3 Интерпретация на AR и Diff кривите

AR панелите показват, че стабилният режим и умерените добавки (KL-keep, Distill) запазват близка и устойчива AR динамика в 90k-121k. Това означава, че тези варианти не разрушават основната next-token способност.

Diff панелите дават по-силен разделителен сигнал:

- при KL-only ($\beta=0$) Diff loss след 90k става нефункционален като сравним индикатор (затова е изключен от zoom Diff панела);
- KL-keep и Distill остават в сравним диапазон със stable, което е знак, че дифузионният клон продължава да учи полезно, докато се добавя alignment.

Изводът е, че пълното изключване на Diff CE може да повиши някои acceptance метрики краткосрочно, но отслабва контрола върху самата Diff реконструкция и прави режима по-рисков за дълги run-ове.

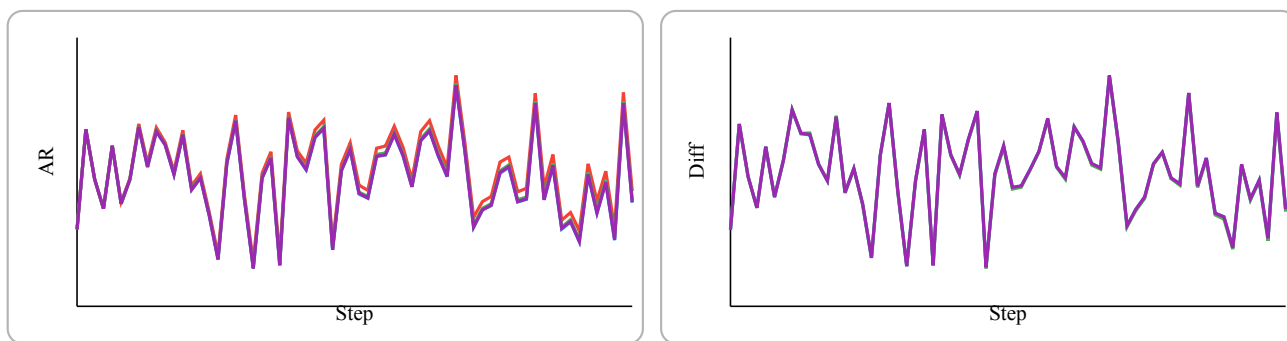


Figure 26: AR и Diff zoom сравнение за 90k-121k

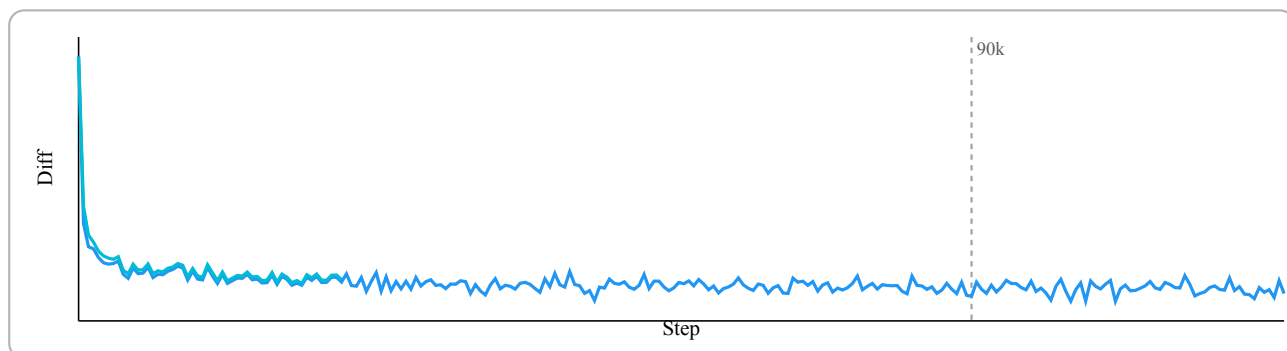


Figure 27: Пълен Diff прозорец: stable срещу ранен delta_masked run

3.9.4 Greedy acceptance: основен таргет на sweep-a

Greedy acceptance е централната метрика в документа ($\text{argmax}(\text{AR}) == \text{argmax}(\text{Diff})$ по валидни позиции). Показаните панели имат три нива на интерпретация:

- absolute панели (90k-121k): сравнение на реалните траектории;
- delta панели спрямо stable: видимост дали вариантът е устойчиво над/под baseline;
- extended панели: добавят агресивни режими и по-късни експериментални хипотези.

Най-важните наблюдения за практиката са:

- KL-keep и Distill дават по-добър баланс между acceptance печалба и запазена training стабилност;
- KL-only може да покаже моментни плюсове в greedy, но на цената на “изключен” Diff обучителен сигнал;
- по-агресивните режими (напр. Big Gamma+Delta) изискват внимателно stage планиране, защото увеличават чувствителността към хиперпараметри;
- delta_masked (късен и ранен вариант) е концептуално обещаващ, защото натиска точно acceptance-критичния префикс, но частичният характер на част от run-овете налага предпазлива интерпретация.

Част от run-овете са прекратявани по-рано, когато се наблюдава силно покачване на AR loss (catastrophic forgetting) или когато Diffusion/Greedy метриците тръгват устойчиво под основната група. Поради това някои криви са по-къси и не достигат до последните точки от общия прозорец. Всички логове и checkpoint-и от тези run-ове са запазени в репото.

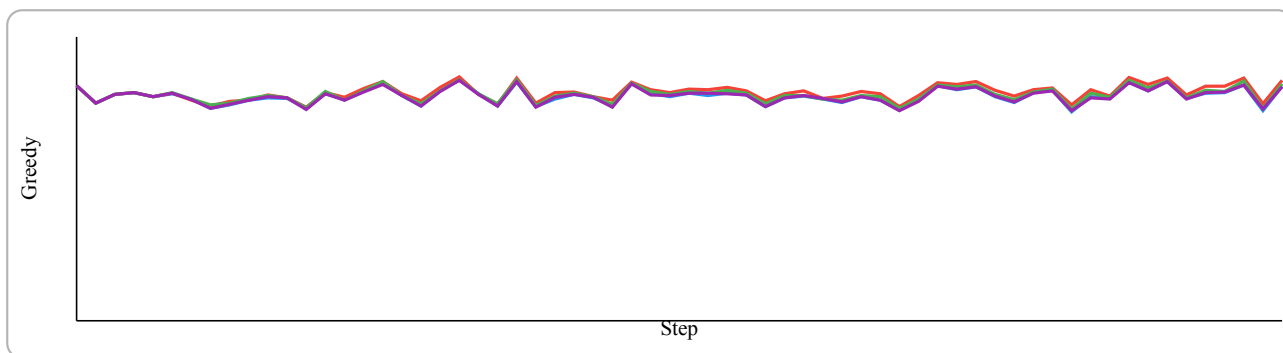


Figure 28: Greedy acceptance: core variants в 90k-121k

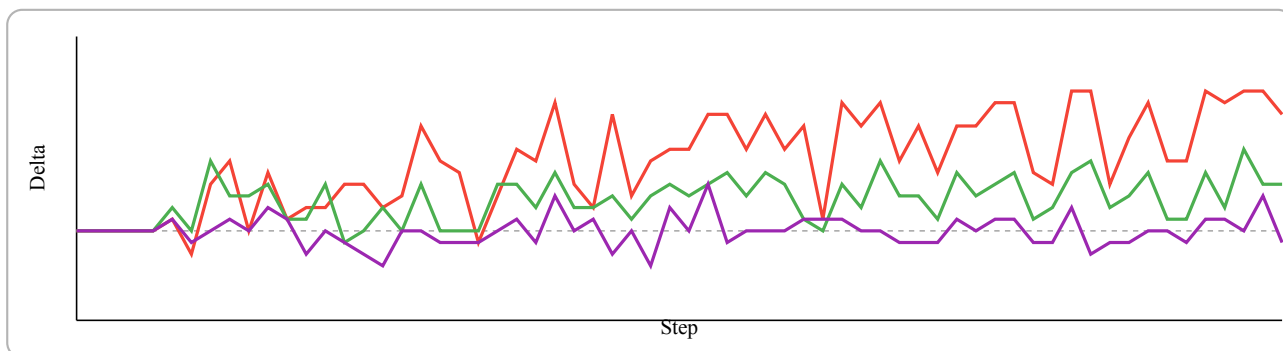


Figure 29: Delta срещу stable за core вариантите

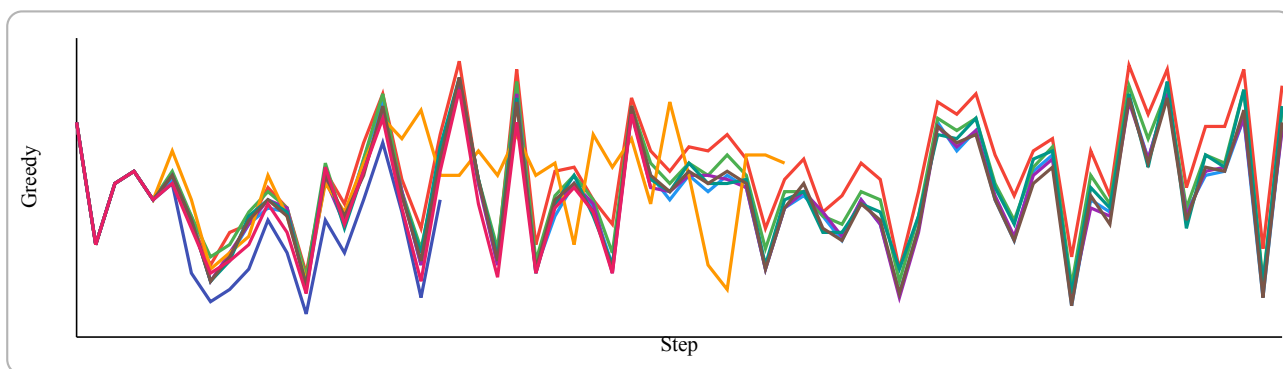


Figure 30: Greedy acceptance: разширен набор от всички варианти

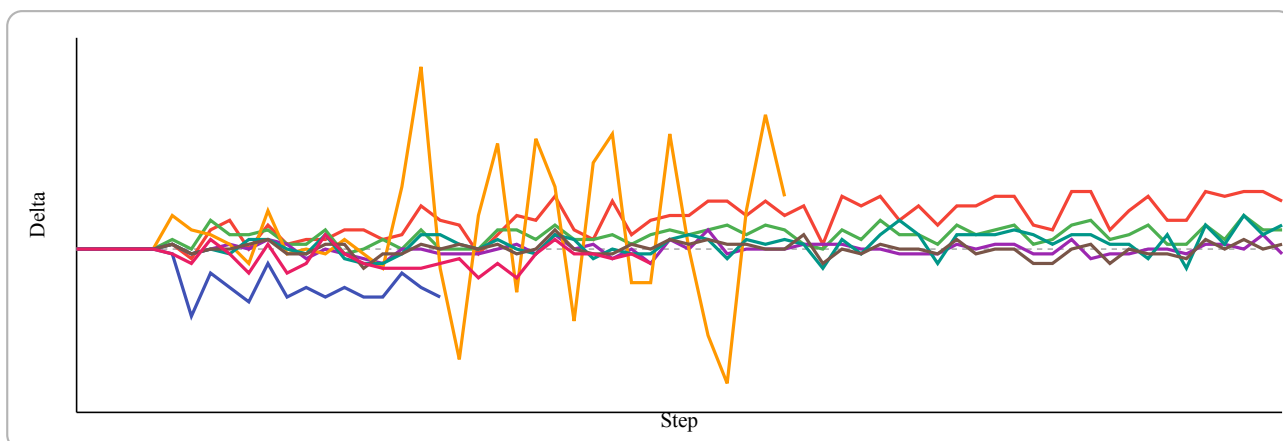


Figure 31: Delta срещу stable за разширения набор

3.9.5 Inference резултати (Асепт/Iter таблица)

Секцията “Inference Results” проверява дали training ефектите се пренасят при реален greedy decode ($\text{temperature}=0$, $\text{draft_len}=6$) върху 10 промпта и три дължини на генериране (50/100/300 стъпки). Ключовите тенденции в таблицата са:

Checkpoint	Steps=50	Steps=100	Steps=300
Stable @90k	2.41 / 3.50 / 1.63	2.26 / 2.75 / 1.71	2.10 / 2.45 / 1.88
Stable @121k	2.22 / 3.27 / 1.53	2.09 / 2.68 / 1.60	2.10 / 2.69 / 1.57
KL-only @121k	2.42 / 3.27 / 1.81	2.30 / 2.68 / 1.80	2.08 / 2.49 / 1.61
KL-keep @121k	2.40 / 3.27 / 1.75	2.29 / 2.83 / 1.80	2.18 / 2.90 / 1.75
Distill @121k	2.31 / 3.50 / 1.53	2.26 / 3.09 / 1.87	2.19 / 2.74 / 1.80
SmallAR+eta @105k	2.15 / 2.88 / 1.75	2.15 / 2.75 / 1.90	2.00 / 2.43 / 1.74
Bigger beta @121.5k	2.37 / 3.27 / 1.63	2.33 / 3.09 / 1.62	1.96 / 2.27 / 1.74
Top-K set @121k	2.28 / 3.12 / 1.52	2.18 / 2.56 / 1.72	2.19 / 2.91 / 1.56
Big Gamma+Delta @105k	2.32 / 3.33 / 1.67	2.26 / 2.86 / 1.79	2.09 / 2.59 / 1.71
Masked Delta later @105k	2.30 / 3.12 / 1.72	2.34 / 2.86 / 1.72	2.01 / 2.40 / 1.67

Table 1: Accept/Iter summary върху 10 TinyStories промпта

- при Steps=300 най-силен среден резултат дават Distill (2.19) и Top-K set (2.19), следвани от KL-keep (2.18), над Stable (2.10);
- при Steps=100 Stable bigger beta достига висока средна стойност (2.33), но при дългия хоризонт (300) пада до 1.96, което подсказва по-слаба устойчивост;
- SmallAR big greedy eta и Masked Delta later са по-агресивни режими и остават под най-добрите устойчиви варианти при по-дълги генерации.

Това потвърждава работната хипотеза, че умереният alignment (KL-keep или Distill) е по-надежден за обща decode ефективност от прекалено силни наказания/насърчения върху отделен компонент.

3.9.6 Локален механистичен анализ чрез logits хистограми

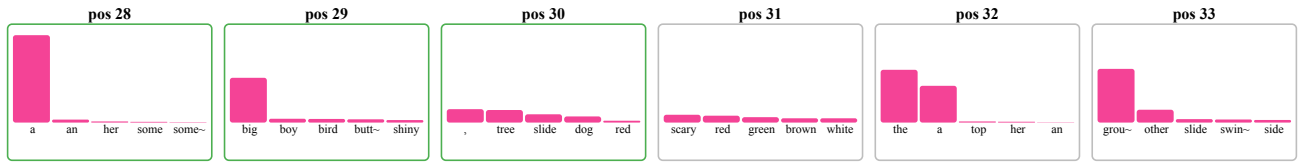
Логитните хистограми дават позиционен анализ на механизма за accept/reject и допълват агрегираните метрики с локална причинна интерпретация. Наблюдава се следният типичен модел:

- в ранните позиции на draft блока AR и Diff често са подравнени по top-1 и токените се приемат;
- около семантично по-нееднозначни позиции Diff става по-разфокусиран или измества top-1 и там започват rejection-и;
- точно тези позиции обясняват защо prefix-ориентирани loss-и (като delta_masked) имат смисъл: те таргетират момента, в който acceptance веригата се прекъсва.

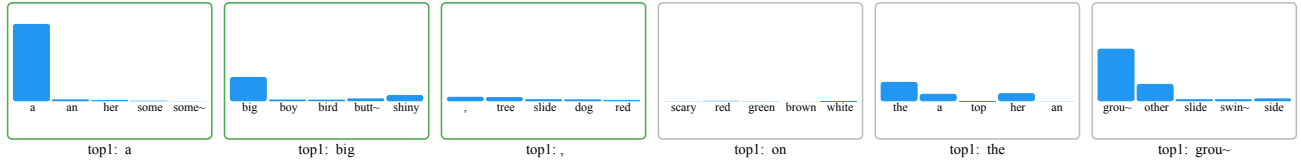
Този тип визуализация е силен аргумент, че sweep-ът не е “black-box” оптимизация по една метрика, а контролирано търсене на причинно обясними подобрения.

Prompt: “Once upon” | Checkpoint: “step_0121566.npz” Target position: 30 | Block: 28-33 | Draft len: 6

AR verify logits (top-5, sorted by AR)



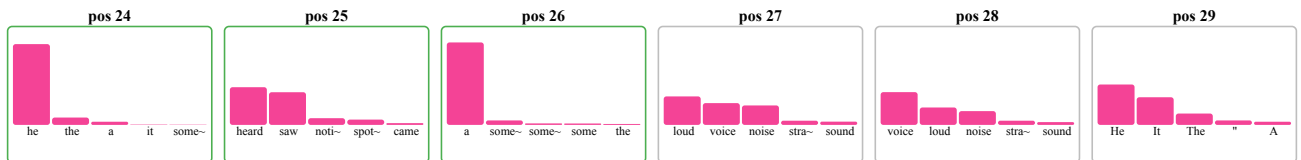
Diff draft logits (same tokens, AR order)



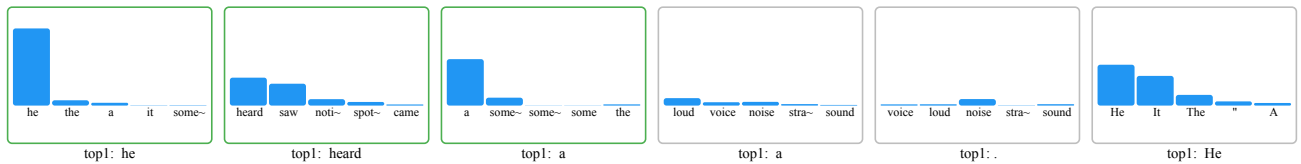
Sentence context One day, she saw a big, scary dog.

Prompt: “In the forest, a tiny fox” | Checkpoint: “step_0121566.npz” Target position: 27 | Block: 24-29 | Draft len: 6

AR verify logits (top-5, sorted by AR)



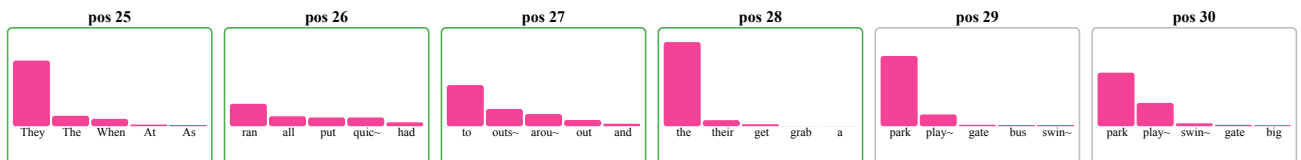
Diff draft logits (same tokens, AR order)



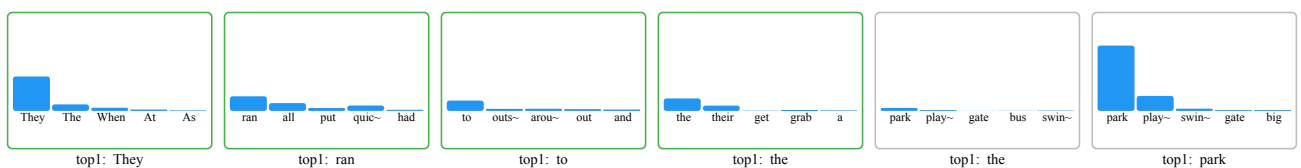
Sentence context Suddenly, he heard a loud noise.

Prompt: “One day, the teacher said” | Checkpoint: “step_0121566.npz” Target position: 29 | Block: 25-30 | Draft len: 6

AR verify logits (top-5, sorted by AR)



Diff draft logits (same tokens, AR order)



Sentence context They ran to the park and saw a big slide.

3.9.7 Финален на експеримента и план за продължение

Проведеният експеримент изпълнява основната си цел: да валидира end-to-end TiDAR training/inference pipeline и да даде първа сравнителна картина за ефекта от различни loss конфигурации. В същото време резултатите показват, че при текущата постановка разделителната способност между вариантите е ограничена.

Основното наблюдение е силното припокриване на кривите между различните директории/конфигурации. В значителна част от диапазона разликите са минимални и на места практически неразличими визуално. Това е индикация, че в текущия режим доминира влиянието на корпуса, а не на конкретния избор на auxiliary loss.

Работната интерпретация е, че TinyStories е нискоентропиен и силно структуриран корпус. При такава среда локална грешка в отделна позиция често не води до трайно разминаване по следващите токени, защото контекстът остава лесен за възстановяване. Поради това режими като delta_masked могат да изглеждат по-слаби в ранния диапазон (0-30k), без това задължително да означава по-нисък потенциал на самия метод в по-труден домейн.

Втори ключов фактор е мащабната разлика спрямо оригиналната експериментална среда на NVIDIA. Тук базовият модел и тренировъчният корпус са приблизително 50 пъти по-малки, докато референтната постановка използва значително по-голям предварително обучен модел (Qwen 1.5B) и много по-широка предтренировъчна база. Това ограничение директно намалява чувствителността на експеримента към фини ефекти от loss комбинациите.

Планирано продължение до защитата (следващи 3 месеца):

- втори експериментален цикъл върху по-информативен корпус от типа OpenWebText/Wikipedia/куриран web text;
- увеличение на модела и тренировъчния обем, така че loss режимите да се разграничат по-ясно;
- избор на най-устойчивите конфигурации от този междинен етап и пренасяне към финален training диапазон от порядъка 300M-1B параметри и 5B-30B токена.

Следователно текущият експеримент се приема като междинен, но необходим етап: системата е валидирана, наблюдавани са ограниченията на малък/лесен корпус, и е дефиниран конкретен план за доразвитие преди финалната защита.

ЧЕТВЪРТА ГЛАВА

РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ

4.1 Инсталация

Базовият начин за стартиране е чрез Docker образите в CICD/Docker/.

Минимален сценарий:

```
docker run --pull always -d --gpus all \
  --name giant-training \
  --mount type=bind,source="$HOME/GIANT",target=/proj \
  bonanc/giant-training:latest
```

```
docker exec -it giant-training bash
```

За automatic-S3-sync workflow е наличен отделен образ CICD/Docker/giant-training-S3/, за който е нужна база като Minio или подобна в облака, за постоянно обновяване на данните. При слаб интернет от страна на developer машината това е неепоръчително. S3 tool-овете са достатъчни за базовия вариант.

4.2 Стартиране на средата и създаване на експеримент

Под “проект” в тази система се разбира комбинация от:

- глобална конфигурация (GIANT/v2/Global_Config.yml, TiDAR/Global_Config.yml);
- локална model/training конфигурация (Config.yml);
- директории за dataset/checkpoints/logs.

Първата стъпка е избор на data root и checkpoint root, след което се изпълняват pipeline и training скриптовете.

4.3 Подготовка и импорт на datasets

Импортиране на datasets.

Примери:

```
python GIANT/v2/data_pipeline/build_corpus.py --config GIANT/v2/data_pipeline/Config.yml
python TiDAR/data_pipeline/Run_pipeline.py --config TiDAR/data_pipeline/data_configs/
Greedy_exp_500m.yml
```

Системата ще генерира Arrow shard-ове и manifest/statistics файлове.

4.4 Работа с training stages

Stage структурата играе ролята на curriculum по време на обучение.

4.4.1 Добавяне и пренареждане на stages

Еквивалент: добавяне и пренареждане на stages в YAML. Всеки stage може да задава seq_len, epochs, fraction и stage-local loss коефициенти.

4.4.2 Избор на checkpoint

Еквивалент: избор на конкретна checkpoint версия за инференс.

```
python GIANT/v2/model/Generate_faster.py --checkpoint latest
python TiDAR/model/inference.py --checkpoint /proj/giant-data/TiDAR/checkpoints/params/
step_0012100.npz
```

4.4.3 Разделяне на run на етапи

Еквивалент: разделяне на training run на етапи с различни objectives или подмяна на config по време на следващ stage.

4.4.4 Нулиране на експеримент

Еквивалент: reset на конкретен експеримент чрез нов checkpoint root или презаписване на stage output директория (pipeline регенерация).

4.5 Управление на изпълнението

Основни команди за runtime управление:

- старт на обучение: `Run_training.py`;
- пауза/спиране: `SIGINT/SIGTERM` (graceful stop след текущ chunk);
- resume: `--resume latest` или път до конкретна checkpoint;
- inference режими: `greedy (--temperature 0)` или `sampling (--temperature > 0, --top_k)`.

4.6 Контрол на KV cache и inference мащаб

За инференс производителност `Generate_faster.py` поддържа KV cache bucket избор:

- автоматичен избор;
- конфигурационни buckets;
- CLI override чрез `--kv_cache_buckets`;
- изключване чрез `--disable_kv_buckets`.

Това позволява баланс между памет и latency, особено при различни prompt/steps комбинации.

4.7 Настройка на training параметри

Настройка на hyperparameters и loss коефициенти.

За TiDAR основните контроли са в `training.loss`:

- `alpha, beta`;
- `rho, chi`;
- `delta, delta_masked`;
- `eta, eta_T`;
- `gamma, gamma_topk`.

4.8 Клавишни комбинации и бързи команди

Работният процес е CLI-first. Типични бързи команди:

- `python GIANT/v2/model/Run_training.py --resume latest`
- `python GIANT/v2/model/Evaluate.py --checkpoint latest`
- `python GIANT/v2/model/Chat.py --chat --steps 128`
- `python GIANT/v2/model/Generate_faster.py --prompt "Hello" --steps 64 --verbose`
- `python TiDAR/model/Run_training.py --config TiDAR/model/Config_135m.yml`
- `python TiDAR/model/inference.py --prompt "Hello" --draft_len 8 --temperature 0.0`

Главният контейнер е базова runtime среда за обучение и инференс, която съдържа само необходимите системни и ML зависимости: CUDA runtime/драйверно-съвместими библиотеки, JAX/Flax/Optax/Orbach стек, Python tooling и build инструменти за възпроизводимо изпълнение на скриптовете. Така се гарантира, че training и inference поведението е консистентно между локална машина и remote GPU среда.

Репото и dataset/checkpoint данните не са вградени в самия Docker образ. Те се добавят след стартиране на контейнера чрез bind mount, sync или S3 workflow. Този подход държи образа по-малък, ускорява обновяването на кода и данните, и избягва ненужно преизграждане на image при всяка промяна в експериментите.

ЗАКЛЮЧЕНИЕ

В настоящата дипломна работа е реализирана цялостна инженерна система за разработка на езиков модел, която обединява data pipeline, обучение, инференс и експериментална инфраструктура в единна codebase.

Основните постижения могат да се обобщят така:

- проектиран е стабилен pipeline за подготовка на данни с shard-ване, manifest и статистика;
- реализирано е обучение на decoder-only Transformer модел с възможност за resume и възстановяване на състояние;
- реализиран е ускорен KV-cache инференс с practically useful CLI сценарии;
- изградена е инфраструктура за remote GPU execution, Docker deployment и S3 синхронизация;
- реализирано е съществено TiDAR разширение с Anchor-TiDAR decode и разширен набор от loss термини за контрол върху acceptance/quality поведението.

Работата показва, че архитектурното разделяне между базова платформа (GIANT/v2) и изследователско разширение (TiDAR) е удачно: базовата част остава стабилна и преизползваема, а експерименталната част може да се развива итеративно без разрушаване на основния workflow.

Възможности за бъдещо развитие:

- добавяне на multi-GPU distributed обучение;
- интеграция на допълнителни архитектурни оптимизации (например MoE/MLA);
- по-богат automated evaluation пакет за quality/speed trade-off;
- допълнително формализиране на експерименталните протоколи и автоматично генериране на отчети.

Исползвана литература

- Vaswani, A. et al. **Attention Is All You Need**. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- Ba, J. L., Kiros, J. R., Hinton, G. E. **Layer Normalization**. 2016. <https://arxiv.org/abs/1607.06450>
- Zhang, B., Sennrich, R. **Root Mean Square Layer Normalization (RMSNorm)**. 2019. <https://arxiv.org/abs/1910.07467>
- Srivastava, N. et al. **Dropout: A Simple Way to Prevent Neural Networks from Overfitting**. 2014. <https://jmlr.org/papers/v15/srivastava14a.html>
- Su, J. et al. **RoFormer: Enhanced Transformer with Rotary Position Embedding**. 2021. <https://arxiv.org/abs/2104.09864>
- Shazeer, N. **GLU Variants Improve Transformer**. 2020. <https://arxiv.org/abs/2002.05202>
- Kingma, D. P., Ba, J. **Adam: A Method for Stochastic Optimization**. 2014. <https://arxiv.org/abs/1412.6980>
- Loshchilov, I., Hutter, F. **Decoupled Weight Decay Regularization (AdamW)**. 2017. <https://arxiv.org/abs/1711.05101>
- Dao, T. et al. **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**. 2022. <https://arxiv.org/abs/2205.14135>
- Dao, T. **FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning**. 2023. <https://arxiv.org/abs/2307.08691>
- Leviathan, Y., Kalman, M., Matias, Y. **Fast Inference from Transformers via Speculative Decoding**. 2023. <https://arxiv.org/abs/2211.17192>
- Stern, M. et al. **Blockwise Parallel Decoding for Deep Autoregressive Models**. 2018. <https://arxiv.org/abs/1811.03115>
- Cai, T. et al. **Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads**. 2024. <https://arxiv.org/abs/2401.10774>
- Li, Y. et al. **EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty**. 2024. <https://arxiv.org/abs/2401.15077>
- NVIDIA Research. **Think in Diffusion, Talk in Autoregression (TiDAR)**. 2025. <https://arxiv.org/pdf/2511.08923>
- Touvron, H. et al. **LLaMA: Open and Efficient Foundation Language Models**. 2023. <https://arxiv.org/abs/2302.13971>
- Hoffmann, J. et al. **Training Compute-Optimal Large Language Models (Chinchilla)**. 2022. <https://arxiv.org/abs/2203.15556>
- Kaplan, J. et al. **Scaling Laws for Neural Language Models**. 2020. <https://arxiv.org/abs/2001.08361>
- Kwon, W. et al. **Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM)**. 2023. <https://arxiv.org/abs/2309.06180>
- NVIDIA. **NVIDIA Ampere Architecture In-Depth**. 2020. <https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/>
- NVIDIA. **NVIDIA A100 Tensor Core GPU Architecture Whitepaper**. 2020. <https://resources.nvidia.com/en-us-tensor-core/nvidia-ampere-architecture-whitepaper>
- NVIDIA. **NVIDIA A40 Datasheet**. 2020. <https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/documents/nvidia-rtx-a40-datasheet.pdf>
- JAX Documentation. <https://docs.jax.dev/>

- Flax Documentation. <https://flax.readthedocs.io/>
- Optax Documentation. <https://optax.readthedocs.io/>
- Orbx Checkpointing Documentation. <https://orbax.readthedocs.io/>
- Hugging Face Datasets Documentation. <https://huggingface.co/docs/datasets>
- Hugging Face Transformers Documentation. <https://huggingface.co/docs/transformers>
- OmegaConf Documentation. <https://omegaconf.readthedocs.io/>
- Docker Documentation. <https://docs.docker.com/>
- Tailscale Documentation. <https://tailscale.com/kb>
- s5cmd Documentation. <https://github.com/peak/s5cmd>
- Andrej Karpathy. **Let's build GPT: from scratch, in code, spelled out.** YouTube, 2023. <https://www.youtube.com/watch?v=kCc8FmEb1nY>
- Andrej Karpathy. **Neural Networks: Zero to Hero** (video playlist). YouTube. <https://karpathy.ai/zero-to-hero.html>
- Andrej Karpathy. **Intro to Large Language Models.** YouTube, 2023. https://www.youtube.com/watch?v=zjkBMFhNj_g
- Typst Documentation. <https://typst.app/docs/>
- DeepSeek-AI. **DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model (MLA + MoE)**, 2024. <https://arxiv.org/abs/2405.04434>
- DeepSeek-AI. **DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models**, 2024. <https://arxiv.org/abs/2401.06066>
- Beltagy, I., Peters, M. E., Cohan, A. **Longformer: The Long-Document Transformer** (sliding-window attention), 2020. <https://arxiv.org/abs/2004.05150>

СЪДЪРЖАНИЕ

ТЕХНОЛОГИЧНО УЧИЛИЩЕ ЕЛЕКТРОННИ СИСТЕМИ	1
ДИПЛОМНА РАБОТА	1
УВОД	2
ПЪРВА ГЛАВА	3
ПРЕГЛЕД НА СЪЩЕСТВУВАЩИ СИСТЕМИ, ПОДОБНИ НА GIANT, И ИЗВЕСТНИ РАЗВОЙНИ СРЕДСТВА	3
1.0 Термини и технологии (кратки дефиниции)	3
LLaMA семейство (Meta)	3
SmolLM / компактни open модели	3
nanoGPT / minGPT	3
Megatron-LM и distributed тренировка	4
vLLM и TensorRT-LLM	4
Python	4
JAX + Flax	4
Optax + Orbx	4
HuggingFace Hub / Datasets / Transformers	4
Docker, Tailscale и S3 инструменти	4
ВТОРА ГЛАВА	6
ФУНКЦИОНАЛНИ ИЗИСКВАНИЯ, АРГУМЕНТАЦИЯ НА ИЗБОРА НА СРЕДСТВА И ФОРМАЛЕН МОДЕЛ	6
2.1 Функционални изисквания	6
2.1.1 Входни източници и ingest pipeline	6
2.1.2 Подготовка на корпус, токенизация и stage-и	6
2.1.3 Конфигурационно управление на експерименти	6
2.1.4 Мониторинг и метрики по време на изпълнение	6
2.1.5 Управление на хиперпараметри и decode политика	7
2.1.6 Артефакти, checkpoint-и и отчетност	7
2.2 Аргументация за избор на езика и софтуерните средства	7
2.2.1 Избор на езика за програмиране	7
2.2.2 Избор на платформа за разработка	7
2.2.3 Избор на фреймуърк за интерфейс	7
2.2.4 Избор на изчислителен фреймуърк	8
2.2.5 Избор на optimizer/checkpoint инструменти	8
2.2.6 Избор на средства за deployment и синхронизация	8
2.3 Структура на данните и вътрешен модел на системата	8
2.3.1 Формат на данните и логика на съхранение	8
2.3.2 Stage конфигурация и curriculum модел	8
2.3.3 Конфигурационни домейни на системата	8
2.3.4 Схема на dataset записите в shard	9
2.3.5 Източници и ingest адаптери	9
2.4 Цялостна архитектура и структурна организация	9
2.4.1 Интерфейсен слой (CLI)	9
2.4.2 Управляващи модули	9
2.4.3 Основни entry points	9
2.5 Основен математически модел на трансформера	10
2.5.1 Теоретична отправна точка: Attention Is All You Need	10
2.5.2 Multi-head attention: защо са нужни “много глави”	11
2.5.3 От класически Transformer към decoder-only LLM	11
2.5.4 Нормализация, активации и позиционна информация в GIANT	12
2.5.5 Обучителна цел (AR) и връзка с TiDAR	13
2.6 Спекулативно декодиране (въведение преди TiDAR)	13

2.6.1	Защо single-user decode не използва добре GPU	14
ТРЕТА ГЛАВА		15
РЕАЛИЗАЦИЯ НА ПРИЛОЖЕНИЕТО		15
3.1	Реализация на потребителски интерфейс	15
3.1.1	Основни входни точки (CLI)	15
3.1.2	Наблюдение и визуализация на резултати	15
3.1.3	Управление на stage прогреса	15
3.1.4	Управление на параметри	15
3.1.5	Добавяне на входни корпуси	15
3.1.6	Експорт на артефакти	15
3.1.7	Примерен training run в терминал	15
3.2	Реализация на основните системни модули	17
3.2.1	ConfigLoaderManager	17
3.2.2	TrainLoopOrchestrator	17
3.2.3	DataLoaderManager	18
3.2.4	CheckpointManager	19
3.2.5	InferenceManager	19
3.2.6	RemoteOpsManager	22
3.3	Реализация на data pipeline	23
3.3.1	Документ като източник на токени	23
3.3.2	Преобразуване от документ към sequence прозорци	24
3.3.3	Директно четене и минимизиране на IO overhead	24
3.3.4	Шардване	24
3.3.5	Loader batch изграждане	24
3.3.6	Допълнителни preprocessing опции	24
3.4	Имплементация на “Think in Diffusion, Talk in AutoRegression”	25
3.4.0.1	Контекст: от класически speculative decode към TiDAR	25
3.4.0.2	Основна идея на TiDAR в един forward pass	25
3.4.0.3	Структурирани маски и достъп по позиции	26
3.4.0.4	Prefill маска	26
3.4.0.5	Training маска	27
3.4.0.6	Loss формулировка и баланс AR:Diff	28
3.4.0.7	Anchor-TiDAR (финален акцент)	28
3.4.1	Представяне на dual sequence вход	28
3.4.2	Преобразуване и маскиране по позиции	29
3.4.3	Извличане на verify и predraft логити	29
3.4.4	KV-cache pointer commit/rollback	29
3.4.5	Decode сценарий (Anchor-TiDAR)	29
3.4.6	Допълнителна визуална интерпретация за M_{ij}	29
3.5	Експеримент: Free Token Slots	30
3.5.1	Native decode attention	30
3.5.2	Kernel terms (контролен анализ)	31
3.5.3	MLP scaling	31
3.5.4	Sampling path (greedy и non-greedy)	32
3.6	Резултати: TiDAR срещу AR (A40 и H100)	32
3.7	Разширена loss функция за TiDAR	33
3.7.1	Формулировка с независими коефициенти	33
3.8	Резултати от greedy run-ове (135M vs 360M)	35
3.9	Проведен експеримент с различни loss функции и конфигурации	35
3.9.0	Избор на TinyStories и базова конфигурация	35
3.9.1	Експериментален протокол и избор на checkpoint	36
3.9.2	Сравнявани loss режими	37

3.9.3 Интерпретация на AR и Diff кривите	37
3.9.4 Greedy acceptance: основен таргет на sweep-a	38
3.9.5 Inference резултати (Accept/Iter таблица)	39
3.9.6 Локален механистичен анализ чрез logits хистограми	40
3.9.7 Финален на експеримента и план за продължение	42
ЧЕТВЪРТА ГЛАВА	43
РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ	43
4.1 Инсталация	43
4.2 Стартиране на средата и създаване на експеримент	43
4.3 Подготовка и импорт на datasets	43
4.4 Работа с training stages	43
4.4.1 Добавяне и пренареждане на stages	43
4.4.2 Избор на checkpoint	43
4.4.3 Разделяне на run на етапи	43
4.4.4 Нулиране на експеримент	44
4.5 Управление на изпълнението	44
4.6 Контрол на KV cache и inference мащаб	44
4.7 Настройка на training параметри	44
4.8 Клавишни комбинации и бързи команди	44
ЗАКЛЮЧЕНИЕ	45
Използвана литература	46